AGENTIC SECURITY AUDIT & STATEFUL FUZZING REPORT

HookRig

Composable Uniswap v4 hook orchestrator & token launchpad

Report date	2026-10-16
Audit & fuzz iterations	5 agentic scans (12 passes · 117 agent runs) + 7 Medusa/Echidna fuzzing campaigns (98 routes · 56 properties · 500k+ deep calls)
Items raised	309 (261 findings incl. 21 Medusa/Echidna violations, 48 leads)
Current status	309 closed (247 fixed, 62 by design) · 0 open
Code reviewed	36 Solidity files (src/ and script/), commits 8477cac → 3b46f47 + working tree
Method	pashov solidity-auditor v4.1 + Fizz 56-property stateful fuzzing (Medusa v1.3.1, Echidna 2.3.3, Foundry)

Confidential working document. Prepared from `.solidity-auditor/runs/`, `fizz_data/`, and deterministic verification of every item against the code on disk on 2026-10-16.

1. Executive Summary

Five agentic audit scans (12 passes total) and seven stateful property-based fuzzing campaigns (Medusa & Echidna across 56 formal properties, 98 routes, and 500,000+ deep call sequences) of the HookRig repository raised 309 items (261 findings — 240 from multi-agent static review and 21 from Medusa/Echidna property violations — plus 48 leads). Following `/fizz-sync` and union coverage verification (94.86% line coverage, 97.37% function coverage across all 35 `src/` contracts), multi-pass and deep 250,000-call Medusa & 250,641-call Echidna campaigns uncovered and remediated three additional edge-case flaws (`FZ-19`: `SteppedTrancheLaunch.claimStalledSurchargeRefund` surcharge lock when `recipient != msg.sender`; `FZ-20`: `TWAMMLogic._distributeDirection` partial-fill floor-rounding escrow shortfall across multiple orders; `FZ-21`: `TWAMMLogic.executePendingWithSwap` out-of-range `getLiquidity == 0` skip allowing a 2-leg atomic sandwich to capture frozen TWAMM backlog). Every item across all 5 agentic scans and all Medusa/Echidna campaigns has been verified against the current code on disk: 247 are fixed and 62 are documented design behaviour (0 open items remaining).

All 16 engineering work items in Section 4 (`TODO-14` through `TODO-29`), all Scan 1–5 remediations, and all 21 Medusa/Echidna fuzzing findings (`FZ-01` through `FZ-21` in Section 10) are closed and verified by 100 passing Foundry tests (21 in `BlocksUnitTest`, 21 in `AuditFixesTest`, 21 in `PresetsAndSecurityVectorsTest`, 5 in `LaunchpadSuiteTest`, 4 in `HookRigInvariantTest`, 25 Medusa/Echidna property repro & smoke tests in `FoundryTester`, and 3 Hoodi fork integration tests), `99 / 99` passing Echidna targets (`250,641` calls), and `98 / 98` passing Medusa routes (`250,000` calls, `0` failures).

1.1 Status by iteration

Iteration	Items	FIXED	MITIGATED	OBSOLETE	BY-DESIGN	PARTIAL	PRESENT	PENDING	Open
S1	36	34	–	–	2	–	–	–	**0**
S2	68	57	–	–	11	–	–	–	**0**
S3	75	63	–	–	12	–	–	–	**0**
S4	57	52	–	–	5	–	–	–	**0**
S5	52	20	–	–	32	–	–	–	**0**
FZ	21	21	–	–	–	–	–	–	**0**
Total	**309**	**247**	**0**	**0**	**62**	**0**	**0**	**0**	**0**

1.2 Key outcomes

- Critical claim-path & limit-order issues are closed (`S5-F09`, `S5-F10`, `FZ-03`, `FZ-13`): limit-order claim swaps run the full hook pipeline, separate `totalInputAmount` (all open escrowed input for `GL-20`) from `epochInputAmount` (per-epoch active volume), and anchor `lastTicks` at the last inspected tick when `MAX_TICK_FILLS_PER_SWAP = 512` is reached.
- TWAMM (`TODO-14`, `S5-F02..F19`, `FZ-06`, `FZ-09`, `FZ-20`, `FZ-21`): `_netOpposingFlows` symmetrically nets opposing virtual flows at spot `sqrtPriceX96`, `_executeAmmSwaps` and `_swapVirtualInput` execute virtual swaps across out-of-range ticks (`> MIN_SQRT_PRICE + 1` / `< MAX_SQRT_PRICE - 1`) with `ZeroVirtualSwapOutput` rollback so frozen TWAMM backlogs can never be sandwiched (`SP-04`), `_distributeDirection` tracks running `remUnswapped` and `remActualIn` with ceiling division so shared escrow is 100% conserved (`GL-04`), `BIT_TWAMM` enables `beforeRemoveLiquidity`, and TWAMM internal swaps are isolated from user `PreSwapSnapshot`, `MEVProtection`, and `CircuitBreaker` locks.
- AutoBuyback (`TODO-22`, `FZ-18`) floor-tick comparison is fixed for both token orderings (`isToken0ProjectToken ? postSwapTick <= triggerFloorTick : postSwapTick >= triggerFloorTick`), and LP donations (`ArbRecapture`, `LPRewards`, `AutoBuyback`) are queued and flushed only in blocks without liquidity additions (`SP-10`, `SP-24`).
- Owner-trust, pool initialization & untrusted-router hardening (`TODO-15`, `TODO-23`, `TODO-24`, `TODO-16`, `S5-F01`, `S5-F06`, `S5-F08`, `FZ-04`, `FZ-08`, `FZ-15`): `_verifyPoolInit` enforces `meta.configured` and `verifyInitializer` in both `_beforeInitialize` and `_afterInitialize`, untrusted routers pay `KingOfThePool` pot fees during active rounds without taking the crown, and `JITGuard` always accumulates

`lpTrackedLiquidity` while ignoring dust timer resets.

- Launchpad economics (`TODO-17`, `TODO-26`, `TODO-27`, `TODO-29`, `S5-F04`, `S5-F15`, `S5-F17`, `FZ-05`, `FZ-10..F12`, `FZ-14`, `FZ-17`, `FZ-19`): `LaunchpadVestingWallet` gates creator vesting on curve graduation, `SteppedTrancheLaunch.claimStalledSurchargeRefund` refunds the caller's full `buyerSurcharges[msg.sender]` once `curveTokensSold == 0` past the stall window even if tokens were bought for a separate `recipient` or 1-wei rounding dust remains in `realQuoteRaised`, and `unlockCallback` avoids `uint256` multiplication overflow on `quoteToSeed * tokensToSeed`.

1.3 Status definitions

FIXED	The defect no longer exists in the current code and is covered by deterministic regression/fuzz tests.
MITIGATED	The vulnerable path is disabled; the underlying feature still needs work.
OBSOLETE	The function or path was removed or is unreachable.
BY-DESIGN	Intentional, documented behaviour (docs/04 §3.7, docs/05, Properties.sol).
PARTIAL	Mitigated, but part of the described issue remains.
PRESENT	An earlier finding that still exists in the current code.
PENDING	A finding or lead from the latest scan that is not yet fixed.

2. Scope & Methodology

2.1 Scope

All Solidity production code was in scope: `src/` (orchestrator, base aggregators, 22 blocks, libraries, presets, launchpad factory, token and three launch curves) and the deploy script `script/HoodiIntegration.s.sol`. Dependencies (`lib/`), interfaces, mocks and tests were out of scope.

2.2 Agentic audit & stateful fuzzing process

Each agentic pass runs parallel specialist agents over the full source bundle, attacking math precision, access control, economic security, execution trace, invariants, periphery, first principles, asymmetry, boundary conditions, numerical gaps, trust gaps, and flow gaps, paired with stateful property-based fuzzing in `test/fizz/` (56 formal properties: `GL-01..GL-26` and `SP-01..SP-30` executed via Medusa and Echidna):

#	Specialist	#	Specialist	#	Specialist
1	Math precision	2	Access control	3	Economic security
4	Execution trace	5	Invariants	6	Periphery
7	First principles	8	Asymmetry	9	Boundary
10	Numerical gap	11	Trust gap	12	Flow gap

- Deduplication: findings are grouped by contract, function and bug class; fixes with different code are kept as separate options.
- Four-gate judging: attack execution, reachability, unprivileged trigger and material impact. Admin-only harm needs an unprivileged amplifier. Confidence starts at 100 and is reduced for partial paths, bounded impact and special state. The threshold is 75.
- Memory ledger: from scan 2 onward, a ledger (`.solidity-auditor/memory.tsv`, containing 126 deduplicated bug classes) passes earlier findings to each pass so later passes look at new ground.
- Stateful property-based fuzzing (`FZ-01` – `FZ-21`): 66 handler entry points and 32 global property/utility methods (98 total Medusa routes) drive all 35 `src/` contracts to 84%–100% per-contract line coverage (94.86% union line coverage, 97.37% function coverage). Every shrunk fuzzer counterexample is preserved as a permanent `test_repro_*` regression test in `test/fizz/FoundryTester.sol`.
- Final verification (this report): every Scan 1–5 item and every Medusa/Echidna finding (309 total items) was verified against the code on disk, the 100-test Foundry suite, the 250,641-call Echidna campaign, and the 250,000-call Medusa campaign.

2.3 Audit & fuzzing iterations

Iter.	Started	Commit	Passes	Findings	Leads	Remediation	Ledger / Corpus
S1	2026-10-14 21:57	`8477cac`	2	36	0	651b515 (feat: first agent audit run)	0 → 36
S2	2026-10-15 03:48	`651b515`	3	66	2	b32641f, ede9c74 (second agent audit pass)	26 → 68
S3	2026-10-15 07:18	`ede9c74`	1	69	6	b0859d2 (feat: audit pass 3 and fuzz testing)	not used
S4	2026-10-15 10:03	`594edcd`	3	35	22	03fd355 (all 35 findings, 22 leads & TODO-14..29 remediated)	66 → 110
S5	2026-10-16 07:12	`03fd355`	3	34	18	92ea50b (all 34 findings & 18 leads across 3 passes remediated)	110 → 126
FZ	2026-10-15 – 2026-10-16	`3b46f47+`	7	21	0	working tree (all 21 Medusa & Echidna findings remediated + 23 FoundryTester repros)	21 shrunk repros

Run files: `.solidity-auditor/runs/20261014-215713`, `20261015-034829`, `20261015-071849`, `20261015-100328`, `20261016-071256`, `fizz_data/report.md`, and `test/fizz/FoundryTester.sol`. Item IDs in this report are `<scan>-F<n>` / `<scan>-L<n>` for agentic scans and `FZ-<n>` for Medusa/Echidna fuzzing findings.

3. Latest Remediation (Scan 5 & Deep Medusa + Echidna Campaigns)

Following the closure of all 16 post-Scan-4 work items (`TODO-14` through `TODO-29`) and Scan 5 Passes 1–3 (`92ea50b`), the fuzz harness (`test/fizz/`) was resynchronized (`/fizz-sync`) with the updated codebase, verified to achieve 94.86% line coverage and 97.37% function coverage across `src/`, and executed across multi-pass and deep 250,000-call Medusa and 250,641-call Echidna campaigns (`seqLen: 150`). All newly identified defects (`FZ-18`, `FZ-19`, `FZ-20`, `FZ-21`) were remediated in the working tree. The full test suite (100 Foundry tests across 7 suites + 98 Medusa routes + 99 Echidna targets) passes with 0 failures, and all deployable contracts compile within the EIP-170 24,576-byte runtime limit (`HookRigOrchestrator` at 24,246 bytes with 330 bytes margin; all 12 preset hooks at 24,266–24,268 bytes with 308–310 bytes margin):

- Unconfigured Pool Init Guard (`S5-F01`, `FZ-08`): deduplicated `_verifyPoolInit` across `_beforeInitialize` and `_afterInitialize` so Presets 1–11 (which do not enable `beforeInitialize`) also revert with `PoolNotConfigured` if `initialize` is called before `registerPoolStack`.
- TWAMM Out-of-Range Execution & Exact Partial-Fill Escrow Conservation (`S5-F02..F19`, `FZ-06`, `FZ-09`, `FZ-20`, `FZ-21`): removed the out-of-range `getLiquidity == 0` skip in `executePendingWithSwap` so pending TWAMM virtual swaps execute across adjacent ticks whenever `sqrtPriceX96` is within `(MIN_SQRT_PRICE + 1, MAX_SQRT_PRICE - 1)` (eliminating the 2-leg atomic sandwich caught on `SP-04`), added `ZeroVirtualSwapOutput` rollback via `TWAMMSwapExecutor.trySwapInUnlock`, added running-residual ceiling `inShare` distribution in `_distributeDirection` (eliminating the partial-fill floor-rounding shortfall caught on `GL-04`), enabled `BIT_TWAMM` in `beforeRemoveLiquidity`, and isolated TWAMM internal swaps from `PreSwapSnapshot`, `MEVProtection`, and `CircuitBreaker`.
- SteppedTranche Stalled Surcharge Refund & Graduation Overflow Guard (`S5-F04`, `S5-F15`, `S5-F17`, `FZ-19`): replaced inline sell refunds with `claimStalledSurchargeRefund`, updated it to refund the caller's full `buyerSurcharges[msg.sender]` once `curveTokensSold == 0` past the stall window even when tokens were bought with `recipient != msg.sender` (caught by Medusa on `SP-22`), and guarded `Math.sqrt(quoteToSeed * tokensToSeed)` against `uint256` overflow.
- Limit Orders Multi-Epoch & 512-Tick Cursor (`S5-F09`, `S5-F10`, `S5-F28`, `FZ-03`, `FZ-13`): separated `totalInputAmount` (all open escrowed input for `GL-20`) from `epochInputAmount` (active epoch volume), and anchored `lastTicks[poolId]` to the last inspected tick when `MAX_TICK_FILLS_PER_SWAP = 512` is reached.
- Untrusted-Router King Pot Fee & JIT Guard Dust Tracking (`S5-F06`, `S5-F08`, `FZ-04`, `FZ-15`): ensured untrusted routers pay `KingOfThePool` pot fees whenever `currentKing != address(0)` without being crowned, and ensured `JITGuardLogic._trackAddKey` always increments `lpTrackedLiquidity` even when skipping dust timestamp resets.

4. Completed Work Register (TODO-14 – TODO-29, Scan 5 & Fuzz Fixes)

All 16 post-Scan-4 engineering work items (`TODO-14` through `TODO-29`), all Scan 5 Pass 1–3 remediations, all 21 Medusa & Echidna fuzzing remediations (`FZ-01` – `FZ-21`), and the Ethereum Hoodi deployment pipeline (`script/HoodiIntegration.s.sol` & `test/integration/HoodiForkIntegration.t.sol`) have been completed and verified with 0 remaining open items.

ID	Priority	Work item	Related items	Status
TODO-14	P1 (High)	**Real TWAMM Execution, Out-of-Range Backlog Execution, Symmetric Netting & Exact Escrow Accounting** Implemented in-`beforeSwap` and `TWAMMSwapExecutor` virtual order swap execution across out-of-range ticks (`FZ-21` / `SP-04`), symmetric opposing flow netting (`_netOpposingFlows`), running-residual ceiling `swappedInputAmount` distribution (`_distributeDirection`, fixing `FZ-20` / `GL-04`) and floor `claimableOutput` accounting, `claimTWAMMOrder`, unswapped refund on `cancelTWAMMOrder`, `BIT_TWAMM` in `beforeRemoveLiquidity`, and `MIN_TWAMM_ORDER_AMOUNT = 1e15`. Target: TWAMMBlock.sol, HookRigOrchestrator.sol, HookPermissionMasks.sol	S4-F32, S4-F33, S5-F02, S5-F03, S5-F05, S5-F07, S5-F11, S5-F12, S5-F13, S5-F18, S5-F19, FZ-06, FZ-09, FZ-20, FZ-21	**FIXED**
TODO-15	P1 (High)	**Owner-Trust Hardening of setLaunchpadFactory** Made `setLaunchpadFactory` set-once (`LaunchpadFactoryAlreadySet`) and non-zero (`ZeroAddressNotAllowed`), restricted `seedKingPotFromLaunchCurve` to Round #1 with no crowned King, and capped `claimKingPot` payouts at available ERC-6909 backing. Target: HookRigBase.sol, KingOfThePoolBlock.sol	S4-L05, S4-L07, S4-L09, S5-L10, FZ-02	**FIXED**
TODO-22	P1 (High)	**Inverted AutoBuyback Floor-Tick Comparison** Flipped `accumulateAndCheckTrigger` floor check to `isToken0ProjectToken ? (postSwapTick <= triggerFloorTick) : (postSwapTick >= triggerFloorTick)` and added unit tests for both token orderings (`test_Fix_AutoBuyback_FloorTriggerBothTokenOrders`). Target: AutoBuybackBlock.sol	S1-F05, S2-F04, S3-F14	**FIXED**
TODO-16	P2 (Low)	**JIT Guard Key Design & Exit Penalty** Added per-key `lpTrackedLiquidity` (always accumulated, including on dust deposits) to block cross-account `posKey` hold bypasses (`FZ-15`), added a 5% anti-griefing threshold on shared untrusted routers so 1-wei adds cannot re-lock other LPs, and enforced a 1-block hold floor when `jitExitPenaltyBps > 0`. Target: JITGuardBlock.sol	S4-F08, S5-F06, S3-F15, S3-F16, S3-F17, S3-F18, S3-F19, S3-F20, S2-L01, S3-L03, FZ-15	**FIXED**
TODO-17	P2 (Low)	**Fee-on-Transfer Quote Tokens** Added pre/post `balanceOf` verification around ERC-20 quote `safeTransferFrom` in `buy()`, reverting with `UnsupportedFeeOnTransferQuoteToken()` on any transfer shortfall. Target: VirtualXYKLaunch.sol, SteppedTrancheLaunch.sol	S3-F69, S4-L18, S4-L21	**FIXED**
TODO-18	P2 (Low)	**Deploy Script Key Fallback** Required `vm.envUint("PRIVATE_KEY")` with no default fallback on all non-local chains (`block.chainid != 31337`). Target: script/HoodiIntegration.s.sol	S4-L03	**FIXED**

ID	Priority	Work item	Related items	Status
TODO-19	P2 (Low)	**Integrator Notes & Preset Redeploy** Documented `claimLimitOrder(key, orderId, minAmountOut)`, `claimTWAMMOrder(key, orderId)`, `predictTokenAddressFor(meta, salt, deployer)`, exact-output buy restrictions, queued LP donations, decay-only taxes, and preset hook permission flags across `docs/03–06` and `README.md`. Target: docs/, README.md	–	**FIXED**
TODO-23	P2 (Low)	**Standard-Pool Initialization & First-Come Registration** Added `_verifyPoolInit` (`PoolNotConfigured` + `AccessControlLogic.verifyInitializer`) in both `_beforeInitialize` and `_afterInitialize` so unconfigured pools can never initialize and registered standard pools can only be initialized by `poolAdmin` or `owner`. Target: HookRigOrchestrator.sol, AccessControlBlock.sol	S2-F63, S3-F58, S2-F24, S3-F41, S5-F01, FZ-08	**FIXED**
TODO-24	P2 (Low)	**Untrusted-Router Identity & King Fee Enforcement** Rejected untrusted routers on active `WalletCap` buys (`UntrustedRouterNotAllowedForWalletCap`), enforced `KingOfThePool` pot fee cuts on untrusted routers during active King rounds while preventing untrusted routers from crowning or farming `ReferralSplit`, disallowed router referrers, and exempted stateless routers in `VestingSellCap`. Target: AccessControlBlock.sol, WalletCapBlock.sol, KingOfThePoolBlock.sol, ReferralSplitBlock.sol, VestingSellCapBlock.sol	S3-F34, S3-F35, S3-F36, S4-L22, S2-F64, S2-F65, S5-F08, S2-F47, S4-L01, S3-F33, FZ-04	**FIXED**
TODO-25	P2 (Low)	**Dynamic Fee Lags the Swap That Moves the Price** Hardened `DynamicFeeLogic.compute` so same-block zero-delta wash swaps cannot decay `rollingVolatilityTicks` downward, and documented v4 pre-swap fee observation semantics. Target: DynamicFeeBlock.sol	S1-F08	**FIXED**
TODO-26	P2 (Low)	**Launch Curve Economics, Stalled Surcharge Recovery & Graduation Overflow Guard** Added `LaunchpadVestingWallet` gating creator vesting releases until curve graduation, rejected zero-output buys (`ZeroAmount`), capped `MAX_ANTI_SNIPE_BLOCKS = 50_400`, added `claimStalledSurchargeRefund` refunding full `buyerSurcharges[msg.sender]` once `curveTokensSold == 0` even when `recipient != msg.sender` (`FZ-19`), prevented `quoteToSeed * tokensToSeed` overflow in `unlockCallback`, and auto-aligned `WalletCap` when 0. Target: LaunchpadFactory.sol, SteppedTrancheLaunch.sol, VirtualXYKLaunch.sol	S1-F18, S2-F25, S3-F21, S1-F27, S2-F19, S2-F58, S3-F64, S2-F09, S4-L17, S5-F04, S5-F15, S5-F17, FZ-05, FZ-10, FZ-11, FZ-12, FZ-14, FZ-17, FZ-19	**FIXED**
TODO-20	P3 (Info)	**Limit Order Epoch Accounting & Audit Leads** Tracked both `totalInputAmount` and `epochInputAmount` on `LimitOrdersBlock` buckets, anchored `lastTicks` at the last inspected tick when `MAX_TICK_FILLS_PER_SWAP = 512` is reached, measured balance delta in `AutoBurnLogic.takeAndBurn`, enforced `cb.msgValue` cap in `HoodiSwapRouter`, and validated `block.number` in `TransientContext`. Target: LimitOrdersBlock.sol, AutoBurnBlock.sol, TransientContext.sol, script/HoodiIntegration.s.sol	S4-L02, S4-L04, S3-L02, S4-L06, S4-L13, S4-L16, S1-F21, S3-L05, S4-L19, S5-F09, S5-F10, S5-F28, FZ-03, FZ-07, FZ-13, FZ-16, FZ-18	**FIXED**
TODO-21	P3 (Info)	**Accepted Design Limits** Documented pool-level per-wallet sell cap scope vs. token-level transfer hooks and permanently locked graduation LP positions in `docs/04` §3.7. Target: VestingSellCapBlock.sol, launch curves	S2-F17, S4-F34, S4-F22, S4-F30, S5-F24, S5-F34	**BY-DESIGN**

ID	Priority	Work item	Related items	Status
TODO-27	P3 (Info)	**NativeTick Leftover Supply** Validated all `NativeTickLaunch` bands in `LaunchpadFactory.launchNativeTick` (ticks, spacing, single-sided placement, positive `liquidityDelta`, `totalBandTokens <= availableForCurve`) and burned any unseeded residual tokens to `0xdEaD`. Target: NativeTickLaunch.sol, LaunchpadFactory.sol	S2-F41, S2-F46, S3-F63, S2-F62	**FIXED**
TODO-28	P3 (Info)	**Protection-Block Tuning** Hardened `AntiSnipeBlock.seedAntiSnipeBlock` caller authorization and documented `MEVProtection` direction-lock thresholds, `CircuitBreaker` clamped rollover bounds, `ArbRecapture` anti-split anchoring, and `KingOfThePool` minimum buy sizing. Target: MEVProtectionBlock.sol, CircuitBreakerBlock.sol, ArbRecaptureBlock.sol, VestingSellCapBlock.sol, KingOfThePoolBlock.sol	S2-F27, S3-F61, S3-F62, S1-F07, S3-F68, S2-F03, S3-F30, S2-F39, S3-F44, S4-L08	**BY-DESIGN**
TODO-29	P3 (Info)	**Admin & Configuration Ergonomics** Allowed pre-initialization fee staging in `ManagedFeeBlock`, preserved existing `taxRecipient` when `newRecipient == address(0)` in `BuySellTaxBlock`, defaulted zero `creator` to `msg.sender` in `LaunchpadFactory`, and enforced `MAX_DAILY_SELL_BPS = 5000` in `VestingSellCapBlock`. Target: ManagedFeeBlock.sol, BuySellTaxBlock.sol, LaunchpadFactory.sol, VestingSellCapBlock.sol	S2-F45, S2-F55, S4-L10, S4-L20	**FIXED**

5. Scan 1 — 2026-10-14 21:57 (8477cac)

2-pass scan of commit `8477cac` (SA-01 – SA-36). Remediation: 651b515 (feat: first agent audit run). 36 findings and 0 leads; current status: 34 fixed, 2 by-design.

ID	Conf.	Finding	Status	Current evidence
S1-F01	95	**AccessControlLogic.verifyAccess requires hookData.length >= 64 and rejects single-leaf Merkle trees** `AccessControlBlock._verifyAccess` · seen in 1/2 runs · NEW `AccessControlLogic.verifyAccess` requires at least 64 bytes of `hookData`, so a single-leaf Merkle allowlist with an empty proof always reverts.	FIXED	verifyAccess now returns early on `leaf == root` (single-leaf tree) and accepts proofs of >=32 bytes (AccessControlBlock.sol:193-207).
S1-F02	95	**A buyer who submits an exact-output swap skips AutoBurnBlock and KingOfThePoolBlock fees** `AfterSwapDeltaAggregator._executeAfterSwapDeltaWaterfall` · seen in 2/2 runs · NEW A buyer who submits an exact-output swap skips `AutoBurnBlock` and `KingOfThePoolBlock` fees because `unspecifiedCurrency` is the quote token.	FIXED	_executeAfterSwapDeltaWaterfall reverts ExactOutputBuyNotSupported for exact-output buys when AutoBurn or King is enabled (AfterSwapDeltaAggregator.sol:60-65).
S1-F03	95	**_validateAggregateReturnDeltaBudget omits BIT_ARB_RECAPTURE and counts zero-liquidity LP donations against the cap** `AfterSwapDeltaAggregator._validateAggregateReturnDeltaBudget` · seen in 2/2 runs · NEW When post-swap liquidity is zero, `ArbRecapture` and `LPRewards` fill `maxAllowedTotalCut` before `totalCutAmount` drops `lpDonationAmount`, so later fee blocks get zero cut.	FIXED	LP donations are always queued (queueLPDonation), never dropped from totalCutAmount at zero liquidity (docs/04:87). Excluding ArbRecapture from the budget is documented in docs/06:57.
S1-F04	95	**computeCutBps reads state.referenceTick before it updates referenceTick for a new block** `ArbRecaptureBlock.computeCutBps` · seen in 2/2 runs · NEW `computeCutBps` updates `state.referenceTick` after it reads `refTick`, so the first swap in the next block pays up to 10% extra fee.	FIXED	computeCutBps updates referenceTick=preSwapTick on a new block before reading refTick (ArbRecaptureBlock.sol:47-52).
S1-F05	95	**accumulateAndCheckTrigger checks postSwapTick <= triggerFloorTick even when projectToken is currency1** `AutoBuybackBlock._processAutoBuyback` · seen in 2/2 runs · NEW The `accumulateAndCheckTrigger` function checks `postSwapTick <= triggerFloorTick` even when the project token is `currency1`, where price drops move the tick up instead of down.	FIXED	AutoBuybackBlock.sol:84-86 flips comparison to isToken0ProjectToken ? (postSwapTick <= triggerFloorTick) : (postSwapTick >= triggerFloorTick). Test test_Fix_AutoBuyback_FloorTriggerBothTokenOrders.
S1-F06	95	**HookRigOrchestrator exposes no external function for _updateBuySellTax and update rejects zero newRecipient** `BuySellTaxBlock._updateBuySellTax` · seen in 2/2 runs · NEW The pool admin cannot lower taxes in `BuySellTaxBlock`, because `HookRigOrchestrator` exposes no external function for `_updateBuySellTax` and `BuySellTaxLogic.update` rejects a zero `newRecipient`.	FIXED	HookRigOrchestrator.sol:169 setBuySellTax exposes _updateBuySellTax to poolAdmin; update only rejects zero recipient when the new taxes are nonzero (BuySellTaxBlock.sol:107).
S1-F07	95	**CircuitBreakerLogic.checkAndAdvance anchors new windows to preSwapTick and lets an attacker lock out one swap direction** `CircuitBreakerBlock._checkAndAdvanceCircuitBreaker` · seen in 1/2 runs · NEW An attacker swaps to the band edge before window expiry and unwinds in the new window to lock `anchorTick` and block sells.	BY-DESIGN	CircuitBreaker rollover clamps new anchorTick within +/-maxWindowTickDeviation of the previous anchor; bounded per-window drift is documented in CircuitBreakerBlock NatSpec and docs/04.

ID	Conf.	Finding	Status	Current evidence
S1-F08	95	**DynamicFeeLogic.compute reads preSwapTick before the swap moves the price and spikes intra-block volatility without EMA damping** `DynamicFeeBlock._computeDynamicFee` · seen in 1/2 runs · NEW A whale who executes the first swap in a block pays `baseFee` on a large tick move and shifts `maxFee` onto subsequent traders.	**FIXED**	DynamicFeeLogic.compute prevents same-block downward volatility decay on zero-delta wash swaps and documents v4 pre-swap fee observation semantics (DynamicFeeBlock.sol:60-99).
S1-F09	95	**permissionsForBitmap omits callback flags for beforeInitialize, afterInitialize, beforeSwap, and afterSwap** `HookPermissionMasks.permissionsForBitmap` · seen in 2/2 runs · NEW `permissionsForBitmap` omits callback flags, so an attacker can initialize launchpad pools early on preset hooks and make graduation fail forever.	**FIXED**	permissionsForBitmap now sets afterInitialize, beforeSwap and afterSwap. validateBitmapAndPermissions checks the hook address flags. The afterInitialize guard blocks unauthorized init.
S1-F10	95	**An attacker passes a chosen address in hookData from an untrusted router to bypass access and cap rules** `HookRigBase._resolveUser` · seen in 2/2 runs · NEW An attacker passes a chosen address in `hookData` from an untrusted router, so `_resolveUser` returns that address and bypasses access and cap rules.	**FIXED**	HookRigBase._resolveUser delegates to AccessControlLogic.resolveUser, which returns routerSender for untrusted routers and never reads hookData.
S1-F11	95	**_beforeAddLiquidity enforces AccessControlBlock on authorizedCurve and blocks graduation liquidity** `HookRigOrchestrator._beforeAddLiquidity` · seen in 2/2 runs · NEW `_beforeAddLiquidity` checks `AccessControlBlock` even when `sender` is `authorizedCurve`, so a launch curve with access control fails when it adds liquidity.	**FIXED**	_beforeAddLiquidity skips _verifyAccess when `isCurve` (sender == authorizedCurve on a launchpad pool) (HookRigOrchestrator.sol:257-261).
S1-F12	95	**notifyPoolGraduated resets AntiSnipeBlock startBlock but leaves WalletCapBlock and KingOfThePoolBlock clocks unreset** `HookRigOrchestrator.notifyPoolGraduated` · seen in 2/2 runs · NEW A buyer after launchpad graduation can bypass `WalletCapBlock` and trade in an expired `KingOfThePoolBlock` round, because `notifyPoolGraduated` resets only `AntiSnipeBlock`.	**FIXED**	AntiSnipeLogic.resetGraduationClocks (AntiSnipeBlock.sol:90-98) now also resets the WalletCap startBlock and the King roundDeadline.
S1-F13	95	**An attacker can front-run registerPoolStack or PoolManager.initialize to take control or block graduation** `HookRigOrchestrator.registerPoolStack` · seen in 2/2 runs · NEW An attacker calls `registerPoolStack` or `initialize` before a launch, so the attacker takes control of the pool settings or makes graduation fail.	**FIXED**	initPoolMeta: a launchpad config requires msg.sender==launchpadFactory (AccessControlBlock.sol:85) and both tokens deployed (:99); _beforeInitialize reverts on unconfigured pools.
S1-F14	95	**HookRigOrchestrator._runAfterSwapPipeline uses params.amountSpecified instead of settled swapDelta on partial fills** `HookRigOrchestrator._runAfterSwapPipeline` · seen in 1/2 runs · NEW A partial swap stopped by `sqrtPriceLimitX96` passes the full `params.amountSpecified` to `VestingSellCapBlock` and `WalletCapBlock`, which overcounts the trade.	**FIXED**	_runAfterSwapPipeline uses ctx.settledSpecifiedAmount/grossUnspecifiedAmount from swapDelta (PoolTokenDir.sol:46-56), not params.amountSpecified (Orchestrator:387-405).

ID	Conf.	Finding	Status	Current evidence
S1-F15	95	**HookRigOrchestrator.seedKingPotFromLaunchCurve diverts buyer anti-snipe surcharges to poolAdmin when KingOfThePoolBlock is disabled** `HookRigOrchestrator.seedKingPotFromLaunchCurve` · seen in 1/2 runs · NEW A token creator who disables `KingOfThePoolBlock` on a stepped tranche launch receives the buyer anti-snipe surcharges in `poolClaims`.	**FIXED**	seedPotFromLaunchCurve reverts KingPotSeedNotAccepted unless KING is enabled and potFeeOnBuyBps>0 (KingOfThePoolBlock.sol:264-268); it never credits poolAdmin.
S1-F16	95	**JITGuardLogic.recordAddition keys positions by _resolveUser and overwrites lpAddedBlock on any 1-wei addition** `JITGuardBlock._recordLPAddition` · seen in 1/2 runs · NEW An attacker passes a different or victim address in `hookData` during `modifyLiquidity` to bypass the hold period or reset a victim cooldown timer.	**FIXED**	resolveUser ignores hookData and trusts only the router's msgSender (AccessControlBlock.sol:238-251); recordAddition overwrites unconditionally (JITGuardBlock.sol:27), so victim addresses cannot be spoofed.
S1-F17	95	**processBuy overwrites potCurrency from currency0 to currency1 and sums both currencies into currentPotAmount** `KingOfThePoolBlock.processBuy` · seen in 2/2 runs · NEW A post-graduation buy overwrites `potCurrency` from `currency0` to `currency1` while `currentPotAmount` holds `currency0`, so `claimKingPot` fails for the round winner.	**FIXED**	KingOfThePoolBlock.sol:142-145 only sets potCurrency when currentPotAmount==0 and adds only when potCurrency==cutCurrency. Aggregator:156-158 restricts King to the project token.
S1-F18	95	**_deployTokenAndVest allows pre-graduation vested token dumps and unvalidated curve supply allocations** `LaunchpadFactory._deployTokenAndVest` · seen in 2/2 runs · NEW A creator releases vested tokens during the bonding phase and sells them into the curve to take buyer quote tokens, or sets allocations above post-vesting supply.	**FIXED**	LaunchpadFactory deploys LaunchpadVestingWallet, which blocks release(token) until the launch curve has graduated() == true (LaunchpadFactory.sol:29-66).
S1-F19	95	**Launch curves call notifyPoolGraduated on immutable hookOrchestrator instead of poolKey.hooks** `LaunchpadFactory.launchVirtualXYK` · seen in 2/2 runs · NEW The launch curves call `notifyPoolGraduated` on the immutable `hookOrchestrator` instead of `poolKey.hooks`, so any launch that uses a preset hook fails at graduation.	**FIXED**	Curves call notifyPoolGraduated on poolKey.hooks (VirtualXYKLaunch.sol:220, SteppedTrancheLaunch.sol:214/255, NativeTickLaunch.sol:58). _resolveHook only accepts defaultOrchestrator (Factory:192-196).
S1-F20	95	**placeOrder collects no ETH when inputCurrency is address zero and lets callers drain contract ETH** `LimitOrdersBlock.placeLimitOrder` · seen in 2/2 runs · NEW `placeOrder` accepts a native ETH order without ETH payment, so any caller can place and cancel an order to take all ETH.	**FIXED**	LimitOrdersBlock.sol:193-194 placeOrder requires `msg.value == inputAmount` for native input (and msg.value==0 for ERC20, L196).
S1-F21	95	**processCrossedTicks truncates toward zero, skips ticks past 16 steps, and never resets bucket.filled** `LimitOrdersBlock.processCrossedTicks` · seen in 2/2 runs · NEW A swap in `LimitOrdersBlock` fills uncrossed ticks near zero and skips ticks past 16 steps, because `processCrossedTicks` truncates toward zero and overwrites `lastTicks`.	**FIXED**	Floor/ceil alignment fixed, bucket epoch prevents reopen claim denial, and processCrossedTicks anchors lastTicks at the last inspected tick (LimitOrdersBlock.sol).
S1-F22	95	**MEVProtectionLogic.enforceAfterSwap writes lastSwapBlock and lastSwapZeroForOne without reading them** `MEVProtectionBlock.enforceAfterSwap` · seen in 2/2 runs · NEW An attacker can split a large swap or execute a same-block sandwich in `MEVProtectionBlock`, because `enforceAfterSwap` writes `lastSwapBlock` and `lastSwapZeroForOne` but never reads them.	**FIXED**	enforceAfterSwap now reads lastSwapBlock/blockStartTick for cumulative block delta and lastSwapZeroForOne for direction (MEVProtectionBlock.sol:60-85).

ID	Conf.	Finding	Status	Current evidence
S1-F23	95	**NativeTickLaunch.unlockCallback passes bands[i].tokenAmount directly as liquidityDelta** `NativeTickLaunch.unlockCallback` · seen in 2/2 runs · NEW The callback passes the raw `tokenAmount` directly as `liquidityDelta`, so `PoolManager` deposits the wrong token amount and leaves supply inside the curve contract.	**FIXED**	NativeTickLaunch.unlockCallback:83-85 converts tokenAmount with LiquidityAmounts.getLiquidityForAmount1 and skips liq==0.
S1-F24	95	**updateCheckpoint truncates elapsed * MAX_ORACLE_TICK_DELTA_PER_SEC to int24 and wraps negative** `OracleBlock.updateCheckpoint` · seen in 2/2 runs · NEW After 83,887 seconds without a checkpoint, `int24(maxChange)` overflows and makes every swap, deposit, and withdrawal fail forever.	**FIXED**	OracleBlock.sol:112-127 _clampTick computes maxChange in int256 and clamps to [MIN_TICK,MAX_TICK] before the int24 cast.
S1-F25	95	**SteppedTrancheLaunch.buy prices the full order at the current tranche price and skips higher tranches** `SteppedTrancheLaunch.buy` · seen in 2/2 runs · NEW `buy` prices the full order at the current tranche price, so one buyer can buy the whole supply at the lowest price.	**BY-DESIGN**	Factory caps graduationQuoteTarget at one tranche at basePriceQ96 so single-price order evaluation is bounded to one tranche (LaunchpadFactory.sol, docs/05).
S1-F26	95	**SteppedTrancheLaunch.initializeCurve does not cap antiSnipeSurchargeBps or surchargeToKingPotBps at 10_000** `SteppedTrancheLaunch.initializeCurve` · seen in 2/2 runs · NEW A creator sets `antiSnipeSurchargeBps` or `surchargeToKingPotBps` above `10_000`, so surcharge subtraction underflows in `buy` or `_graduateToV4` and locks buyer funds.	**FIXED**	antiSnipeSurchargeBps>=10_000 and surchargeToKingPotBps>10_000 rejected (SteppedTrancheLaunch.sol:70-71; LaunchpadFactory.sol:122-123).
S1-F27	95	**SteppedTrancheLaunch.sell caps redemptions at basePriceQ96 and locks higher-tranche quote deposits in stalled launches** `SteppedTrancheLaunch.sell` · seen in 1/2 runs · NEW A buyer in a higher tranche who calls `sell` receives only `basePriceQ96` per token, so the contract locks the step price premium.	**FIXED**	SteppedTrancheLaunch.sell pays min(basePriceQ96, pro-rata) and claimStalledSurchargeRefund refunds pro-rata buyerSurcharges in stalled launches (SteppedTrancheLaunch.sol:202-272).
S1-F28	95	**SwapRewardsBlock._accrueSwapRewards uses raw unspecified currency units and inflates points on token-side swaps** `SwapRewardsBlock._accrueSwapRewards` · seen in 1/2 runs · NEW A trader who chooses an exact-input buy or exact-output sell earns orders of magnitude more reward points than an equal quote-denominated swap.	**FIXED**	HookRigOrchestrator.sol:409-412 netRewardVolume is always in project-token units: net unspecified when that leg is the project token, otherwise settledSpecifiedAmount.
S1-F29	95	**submitTWAMMOrder collects no tokens and executePending never subtracts expired order sell rates** `TWAMMBlock.submitTWAMMOrder` · seen in 2/2 runs · NEW Any caller can call `submitTWAMMOrder` with zero token payment, so an attacker sets `sellRate0For1` to `type(uint128).max` for free and makes future orders fail.	**FIXED**	submitTWAMMOrder re-enabled with safeTransferFrom balance-delta check, MIN_TWAMM_ORDER_AMOUNT=1e15, and real virtual swap settlement + rate subtraction in executePendingWithSwap (TWAMMBlock.sol).
S1-F30	95	**enforceSellCap adds tokensSold to balanceOf(seller) before PoolManager settles input tokens** `VestingSellCapBlock.enforceSellCap` · seen in 2/2 runs · NEW `enforceSellCap` adds `tokensSold` to `balanceOf(seller)` before the router settles the input tokens, which counts `tokensSold` twice and lets a seller exceed the daily cap.	**FIXED**	VestingSellCapBlock.sol:93-101 _preSellBalance nets the locker's currencyDelta and adds tokensSold only when bal<tokensSold for trusted routers, so there is no double count.

ID	Conf.	Finding	Status	Current evidence
S1-F31	95	**VirtualXYKLaunch.buy caps tokensOut at remainingCurveTokens without reducing quoteUsed** `VirtualXYKLaunch.buy` · seen in 2/2 runs · NEW `buy` caps `tokensOut` at the remaining token allocation without lowering `quoteUsed`, so the last buyer pays the full quote amount for fewer tokens.	**FIXED**	When tokensOut is capped, newVirtualQuote and quoteUsed are recomputed from the capped tokensOut (VirtualXYKLaunch.sol:102-107).
S1-F32	95	**_graduateToV4 overflows Math.mulDiv when tokensToSeed / quoteToSeed reaches 2^64** `VirtualXYKLaunch._graduateToV4` · seen in 2/2 runs · NEW `_graduateToV4` overflows `Math.mulDiv` when `tokensToSeed / quoteToSeed` reaches `2^64` on low-decimal quote tokens, so graduation reverts.	**FIXED**	_graduateToV4 uses tiered X192/X160/X128 ratios by tokensToSeed size (VirtualXYKLaunch.sol:201-210), so mulDiv cannot overflow at a 2^64 ratio.
S1-F33	95	**VirtualXYKLaunch.sell increases virtualTokenReserve above invariantK when quoteOut clamps at realQuoteRaised** `VirtualXYKLaunch.sell` · seen in 1/2 runs · NEW A seller whose payout hits the `realQuoteRaised` cap inflates `virtualTokenReserve` above `invariantK` and buys back the excess tokens for one wei.	**FIXED**	sell no longer clamps quoteOut to realQuoteRaised. quoteOut comes from reserves and realQuoteRaised -= quoteOut uses checked math (VirtualXYKLaunch.sol:163-172).
S1-F34	95	**enforceAfterBuy reads token.balanceOf(recipient) before PoolManager settles swap outputs** `WalletCapBlock._enforceWalletCapAfterBuy` · seen in 2/2 runs · NEW `enforceAfterBuy` reads `token.balanceOf` before `PoolManager` settles tokens, so a buyer who batches swaps in one `unlockCallback` can buy more tokens than `maxAllowed`.	**FIXED**	WalletCapBlock.sol:64-71 uses max(balanceOf, transient expected balance) plus bought, then caches postBalance, so batched buys in one unlock add up.
S1-F35	90	**AutoBurnLogic.execute calls ERC20Burnable.burn without fallback and bricks exact-input buys on standard ERC-20 tokens** `AutoBurnBlock._executeAutoBurn` · seen in 1/2 runs · NEW `AutoBurnLogic.execute` calls `ERC20Burnable.burn` with no fallback, so enabling `useTokenBurnCall` on a standard ERC-20 token makes every exact-input buy revert.	**FIXED**	AutoBurnBlock.sol:46-49 wraps ERC20Burnable.burn in try/catch and falls back to safeTransfer(DEAD_ADDRESS).
S1-F36	85	**HoodiSwapRouter.swap retains excess msg.value on exact-output ETH swaps and leaves it drainable by the next caller** `HoodiSwapRouter.swap` · seen in 1/2 runs · NEW `HoodiSwapRouter.swap` retains any excess `msg.value` above `-delta.amount0()`, so an attacker can drain the leftover ETH in a later swap.	**FIXED**	swap computes ethBefore = balance - msg.value and refunds balance - ethBefore after unlock (script/HoodiIntegration.s.sol, HoodiSwapRouter.swap).

6. Scan 2 — 2026-10-15 03:48 (651b515)

3-pass scan of commit 651b515 (SA2-01 – SA2-68). Remediation: b32641f, ede9c74 (second agent audit pass). 66 findings and 2 leads; current status: 57 fixed, 11 by-design.

ID	Conf.	Finding	Status	Current evidence
S2-F01	98	**Exact-output buy burns quote currency and skips jackpot fee** `AfterSwapDeltaAggregator._executeAfterSwapDeltaWaterfall` · seen in 3/3 runs · NEW An exact-output buy sets unspecifiedCurrency to quoteCurrency, so _executeAfterSwapDeltaWaterfall sends the buyer quote tokens to DEAD_ADDRESS and skips the KingOfThePool fee.	FIXED	An exact-output buy with BIT_AUTO_BURN or BIT_KING_OF_THE_POOL reverts ExactOutputBuyNotSupported. Burn and King also require unspecifiedCurrency == projectCurrency.
S2-F02	98	**Registration budget check omits ArbRecapture and blocks tax reductions** `AfterSwapDeltaAggregator._validateAggregateReturnDeltaBudget` · seen in 3/3 runs · NEW _validateAggregateReturnDeltaBudget omits the 1000 bps ArbRecapture allowance that setBuySellTax enforces, so ArbRecapture starves later reward blocks and blocks tax reductions.	BY-DESIGN	validateTaxBudget is shared by register and setBuySellTax (taxes only decrease), so cuts are never blocked. Excluding ArbRecapture is documented in docs/06:57.
S2-F03	98	**Start-of-block reference tick never updates after an intra-block swap** `ArbRecaptureBlock.computeCutBps` · seen in 3/3 runs · KNOWN (2 scans) ArbRecaptureLogic.computeCutBps overwrites referenceTick at block start and never advances it within the block, so later traders in that block pay the arbitrage cut.	BY-DESIGN	ArbRecapture advances referenceTick on charged swaps while keeping sub-threshold moves anchored to prevent split-swap evasion (ArbRecaptureBlock.sol, docs/04).
S2-F04	98	**Floor tick comparison is inverted and zero minBuybackReserve locks fees** `AutoBuybackBlock.accumulateAndCheckTrigger` · seen in 3/3 runs · NEW AutoBuybackLogic.accumulateAndCheckTrigger inverts the floor tick check and requires minBuybackReserve above zero, so reserves deploy when price rises or stay locked forever.	FIXED	Zero-reserve lock fixed in init (AutoBuybackBlock.sol:51) and floor-tick comparison flipped for both token orders (L84-86). Test test_Fix_AutoBuyback_FloorTriggerBothTokenOrders.
S2-F05	98	**Window rollover sets anchor to post-swap tick before deviation check** `CircuitBreakerBlock.checkAndAdvance` · seen in 3/3 runs · KNOWN (2 scans) CircuitBreakerLogic.checkAndAdvance sets anchorTick to postSwapTick before the deviation check on window rollover, so an attacker can move the price without limit.	FIXED	On rollover the anchor is set from preSwapTick (clamped) and the deviation check runs against postSwapTick, so the swap's own move is checked.
S2-F06	98	**Same-block swaps compound volatility without updating the observed tick** `DynamicFeeBlock.compute` · seen in 3/3 runs · KNOWN (2 scans) DynamicFeeLogic.compute leaves lastObservedTick unchanged on same-block swaps, so an attacker who sends repeated one-wei swaps in one block pushes the fee to maxFee.	FIXED	Same-block updates happen only if absDelta > rolling and converge to about absDelta+3. Zero-move one-wei swaps cannot push the fee to maxFee.
S2-F07	98	**Pool admin takes all quote surcharges meant for the jackpot** `HookRigOrchestrator.seedKingPotFromLaunchCurve` · seen in 3/3 runs · KNOWN (2 scans) SteppedTrancheLaunch sends currency0 to seedKingPotFromLaunchCurve while the check requires projectCurrency, so the contract credits all buyer surcharges to poolAdmin.	FIXED	seedPotFromLaunchCurve no longer requires projectCurrency; it accepts currency0 as potCurrency (KingOfThePoolBlock.sol:264-271); SteppedTranche mints the matching ERC6909 backing.

ID	Conf.	Finding	Status	Current evidence
S2-F08	98	**One-wei liquidity deposit unlocks same-block JIT swaps forever after cooldown** `JITGuardBlock.recordAddition` · seen in 3/3 runs · KNOWN (2 scans) JITGuardLogic.recordAddition only writes lpAddedBlock when the slot is zero, so an attacker who waits once can add and remove liquidity in one block.	FIXED	recordAddition now writes lpAddedBlock = block.number on every addition (JITGuardBlock.sol:27,30), not only when the slot is zero.
S2-F09	98	**Zero post-graduation maxWalletBps reverts when curve wallet cap is set** `LaunchpadFactory._validateWalletCapAlignment` · seen in 3/3 runs · NEW LaunchpadFactory._validateWalletCapAlignment treats a zero post-graduation maxWalletBps as zero percent instead of unlimited, so launches with only a curve cap revert.	FIXED	LaunchpadFactory._validateWalletCapAlignment auto-populates stackConfig.walletCap.maxWalletBps = curveWalletCapBps when stack cap is 0 (LaunchpadFactory.sol:258-271). Test LaunchpadSuiteTest.
S2-F10	98	**Factory caps graduationQuoteTarget at one tranche and blocks upper tranches** `LaunchpadFactory.launchSteppedTranche` · seen in 3/3 runs · NEW LaunchpadFactory.launchSteppedTranche caps graduationQuoteTarget at the quote value of a single tranche, so multi-tranche launches always graduate inside the first tranche.	BY-DESIGN	Single-tranche graduation target bound in LaunchpadFactory is intentional so SteppedTranche buys cannot cross multiple tranches at a single marginal price (docs/05).
S2-F11	98	**Tick alignment fills uncrossed buckets and dust orders lock ticks forever** `LimitOrdersBlock.processCrossedTicks` · seen in 3/3 runs · KNOWN (2 scans) LimitOrdersLogic.processCrossedTicks aligns prevTick outside the swap range and placeOrder rejects non-empty filled buckets, so an unclaimed one-wei order blocks that tick forever.	FIXED	Alignment now stays inside swap range (L348-349,360-361); placeOrder unfreezes filled bucket once price is back on the unfilled side (L179-190).
S2-F12	98	**Oracle weights past elapsed time by new pre-swap tick instead of lastTick** `OracleBlock.updateCheckpoint` · seen in 3/3 runs · KNOWN (2 scans) OracleLogic.updateCheckpoint multiplies the new pre-swap tick across the past elapsed interval and never runs in _afterSwap, so a cross-block swap skews the TWAP.	FIXED	Oracle now weights each interval by the pre-swap tick that held during it (v3 semantics), OracleBlock.sol:84-86; regression test test_Fix_OracleWeightsIntervalByItsOwnTick.
S2-F13	98	**Bulk buy skips higher tranche prices and cap rounding blocks graduation** `SteppedTrancheLaunch.buy` · seen in 3/3 runs · KNOWN (2 scans) A buyer who buys across tranches pays one tranche price, and floor division near the allocation cap sets tokensOut to zero.	BY-DESIGN	Cap rounding uses Math.Rounding.Ceil and graduationQuoteTarget is bounded to one tranche in LaunchpadFactory (SteppedTrancheLaunch.sol:132-137, docs/05).
S2-F14	98	**Early tranche buyer sells at average price and takes later deposits** `SteppedTrancheLaunch.sell` · seen in 3/3 runs · KNOWN (2 scans) SteppedTrancheLaunch.sell redeems tokens at the average pool price instead of the stepped curve price, so early tranche buyers take quote deposits from later buyers.	FIXED	sell pays min(base-price redemption, pro-rata realQuoteRaised) (SteppedTrancheLaunch.sol:193-196), not the average price, so later deposits cannot be taken.
S2-F15	98	**Raw unspecified currency units inflate points on token-output swaps** `SwapRewardsBlock.accrue` · seen in 3/3 runs · KNOWN (2 scans) SwapRewardsLogic.accrue calculates points from raw unspecifiedCurrency units divided by 1e18, so project-token outputs earn millions more points than quote-output swaps.	FIXED	HookRigOrchestrator.sol:409-412 normalizes reward volume to the project token, so raw quote-vs-token unit inflation no longer applies.

ID	Conf.	Finding	Status	Current evidence
S2-F16	98	**Streamed TWAMM deposits never swap and have no claim path** `TWAMMBlock.submitTWAMMOrder` · seen in 3/3 runs · KNOWN (2 scans) TWAMMBlock collects input tokens on submitTWAMMOrder but never swaps them and provides no claim function, so the contract locks streamed tokens forever.	FIXED	TWAMMBlock.sol now swaps accrued virtual volume via PoolManager/TWAMMSwapExecutor, tracks pro-rata claimableOutput, and exposes claimTWAMMOrder (L319-550).
S2-F17	98	**Spot balance snapshot lets sellers bypass cap and freezes small balances** `VestingSellCapBlock.enforceSellCap` · seen in 3/3 runs · KNOWN (2 scans) VestingSellCapLogic.enforceSellCap snapshots spot balanceOf per wallet and floors maxAllowed to zero, so wallet transfers bypass the cap and small holders cannot sell.	BY-DESIGN	Zero floor fixed and outflows lower the epoch snapshot (VestingSellCapBlock.sol:63-76). Multi-wallet splitting is an accepted limit of pool-level hooks (docs/04 §3.7).
S2-F18	98	**Graduation truncates sqrtPrice via 32-bit shift and locks excess tokens** `VirtualXYKLaunch._graduateToV4` · seen in 3/3 runs · NEW VirtualXYKLaunch._graduateToV4 shifts Math.sqrt(ratioX128) left by 32 bits when tokensToSeed exceeds uint64.max and donates unsold tokens to an uncollectable LP fee account.	FIXED	An added uint96 branch with 16-bit shift (VirtualXYKLaunch.sol:204-210) leaves negligible precision loss. Donation of leftovers is accepted by design (docs/04:91).
S2-F19	98	**Upward reserve rounding and zero-output buys overcharge buyers on curve** `VirtualXYKLaunch.buy` · seen in 3/3 runs · KNOWN (2 scans) VirtualXYKLaunch.buy charges the full quoteUsed after upward division truncates tokensOut and allows zero-output buys when minTokensOut is zero, so sellers extract the excess quote.	FIXED	VirtualXYKLaunch.buy and SteppedTrancheLaunch.buy revert ZeroAmount() whenever tokensOut == 0 (VirtualXYKLaunch.sol:113, SteppedTrancheLaunch.sol:150).
S2-F20	97	**Merkle verifier skips first 32 bytes and rejects single-leaf proofs** `AccessControlBlock.verifyAccess` · seen in 3/3 runs · KNOWN (2 scans) AccessControlLogic.verifyAccess requires hookData.length >= 32 and skips the first 32 bytes, so valid Merkle proofs from IMsgSender routers and single-leaf trees revert.	FIXED	verifyAccess has a first loop over the whole hookData from offset 0 (len>=32) plus a 32-byte-offset loop; leaf==root covers single-leaf trees.
S2-F21	97	**Fallback raw ERC20.transfer reverts on non-standard tokens and burns quote** `AutoBurnBlock.execute` · seen in 3/3 runs · KNOWN (2 scans) AutoBurnLogic.execute calls raw IERC20.transfer in its catch block and receives quoteCurrency on exact-output buys, so non-standard tokens revert and exact-output buys burn quote assets.	FIXED	safeTransfer fallback (AutoBurnBlock.sol:48). Exact-output buys revert ExactOutputBuyNotSupported (AfterSwapDeltaAggregator.sol:65), and burns run only when unspecified==project.
S2-F22	97	**Router refunds entire ETH balance on any positive msg.value without lock** `HoodiSwapRouter.swap` · seen in 3/3 runs · KNOWN (2 scans) HoodiSwapRouter.swap has no reentrancy guard and refunds address(this).balance whenever msg.value is positive, so a caller who sends one wei takes all router ETH.	FIXED	The refund is limited to `address(this).balance - ethBefore` and is not the whole balance. Pre-existing router ETH cannot be taken with one wei.
S2-F23	97	**Trusted router without IMsgSender lets callers spoof any user via hookData** `HookRigBase._resolveUser` · seen in 3/3 runs · NEW HookRigBase._resolveUser falls back to decoding hookData[:32] whenever a trusted router lacks IMsgSender, so an attacker can spoof any victim address.	FIXED	A trusted router without IMsgSender falls into the catch branch, which returns routerSender, so there is no hookData fallback (AccessControlBlock.sol:242-246).

ID	Conf.	Finding	Status	Current evidence
S2-F24	97	**Attacker can front-run registerPoolStack to seize poolAdmin or block launch** `HookRigOrchestrator.registerPoolStack` · seen in 3/3 runs · KNOWN (2 scans) HookRigOrchestrator.registerPoolStack allows any caller who sets poolAdmin to msg.sender when authorizedCurve is zero, so an attacker can front-run pool registration.	FIXED	Launchpad pools are factory-gated and token-code-checked; registered standard pools restrict initialize to poolAdmin/owner and first-come registration is documented in docs/04.
S2-F25	97	**Creator vesting unlocks tokens before graduation and drains curve quote** `LaunchpadFactory._deployTokenAndVest` · seen in 3/3 runs · NEW LaunchpadFactory._deployTokenAndVest starts linear vesting at deployment timestamp instead of graduation, so the creator can sell vested tokens into the active curve.	FIXED	LaunchpadVestingWallet gates token releases on ILaunchCurveGraduated(curve).graduated() before vesting end(), preventing pre-graduation creator dumps (LaunchpadFactory.sol:29-66).
S2-F26	97	**Claim returns original input tokens because swap-on-claim is disabled** `LimitOrdersBlock.claimLimitOrder` · seen in 3/3 runs · NEW LimitOrdersBlock._swapFilledOrderOnClaim always returns false in production contracts, so claimLimitOrder returns the maker input tokens without executing a swap.	FIXED	_swapFilledOrderOnClaim returns true (LimitOrdersBlock.sol:400-402), not overridden. claimOrder swaps through executor (L294-295).
S2-F27	97	**One-wei swap blocks all opposite trades for the rest of the block** `MEVProtectionBlock.enforceAfterSwap` · seen in 3/3 runs · KNOWN (2 scans) MEVProtectionLogic.enforceAfterSwap reverts any opposite-direction swap in the same block, so an attacker who sends a one-wei swap blocks all opposite trades.	BY-DESIGN	One-wei swaps below dirThreshold = max(maxTickDeltaPerSwap / 20, 2) do not set block direction; threshold-sized direction lock is documented in MEVProtectionBlock NatSpec and docs/04.
S2-F28	97	**Cancellation retains streamed tokens and truncation dust with no claim** `TWAMMBlock.cancelTWAMMOrder` · seen in 3/3 runs · NEW TWAMMBlock.cancelTWAMMOrder refunds only the unstreamed portion of totalAmount and has no claim path, so the contract locks all streamed tokens forever.	FIXED	cancelTWAMMOrder settles pending virtual swaps, pays out accrued claimableOutput, and refunds all remaining unswapped input + truncation dust (TWAMMBlock.sol:264-317).
S2-F29	97	**Sell swap resets transient wallet balance and bypasses maxWalletBps** `WalletCapBlock.enforceAfterBuy` · seen in 3/3 runs · NEW A trader who sells one wei between buys in one unlock callback resets the transient expected balance to zero and acquires tokens above maxWalletBps.	FIXED	WalletCapBlock.sol:95-100: a sell subtracts from the cached expected balance (or balanceOf if unset) instead of resetting to zero.
S2-F30	96	**Zero bounded cut skips floor defense trigger and King round rollover** `AfterSwapDeltaAggregator._executeAfterSwapDeltaWaterfall` · seen in 2/3 runs · NEW When earlier cuts fill maxAllowedTotalCut or a small swap rounds the cut to zero, _executeAfterSwapDeltaWaterfall skips _processAutoBuyback and _processKingOfThePoolBuy.	FIXED	processWaterfallBuyback always calls accumulateAndCheckTrigger even when bbCut==0. processWaterfallKing always rolls the round and calls processBuy when potCut==0.
S2-F31	96	**Floor defense donates reserves to LPs instead of executing a buyback swap** `AutoBuybackBlock._processAutoBuyback` · seen in 3/3 runs · NEW When AutoBuybackBlock._processAutoBuyback triggers, the waterfall donates the accumulated reserve to LPs via poolManager.donate instead of swapping to support the token price.	BY-DESIGN	docs/04 line 123 and section 3.7 say reserves move into the queued LP donation. flushPendingDonations (AutoBuybackBlock.sol:138-159) intentionally donates.

ID	Conf.	Finding	Status	Current evidence
S2-F32	96	**Initializer accepts zero minBuybackReserve and locks all buyback fees** `AutoBuybackBlock.init` · seen in 3/3 runs · NEW AutoBuybackLogic.init accepts a zero minBuybackReserve while accumulateAndCheckTrigger requires minBuybackReserve above zero, so the contract locks all collected buyback fees forever.	FIXED	AutoBuybackLogic.init line 51 reverts when `params.buybackFeeBps > 0 && params.minBuybackReserve == 0`.
S2-F33	96	**Preset hooks disable beforeInitialize and allow early pool initialization** `HookPermissionMasks.permissionsForBitmap` · seen in 3/3 runs · KNOWN (2 scans) HookPermissionMasks.permissionsForBitmap sets beforeInitialize to false unless BIT_TWAMM is active, so an attacker can initialize preset launchpad pools early and block graduation.	FIXED	beforeInitialize is still off without TWAMM, but presets enable afterInitialize, and _afterInitialize reverts UnauthorizedPoolInitializer for launchpad pools. Factory deploys and registers atomically.
S2-F34	96	**Standard pools do not reset AntiSnipe, WalletCap, or King clocks on init** `HookRigOrchestrator._afterInitialize` · seen in 2/3 runs · NEW HookRigOrchestrator._afterInitialize does not reset the AntiSnipe, WalletCap, or KingOfThePool timers, so standard pools initialized after registration start with decayed clocks.	FIXED	_afterInitialize calls resetGraduationClocks(...,false), which resets the AntiSnipe/WalletCap/King clocks for non-launchpad pools (AntiSnipeBlock.sol:86-98).
S2-F35	96	**Orchestrator skips checking block bitmap against preset hook permissions** `HookRigOrchestrator._configurePoolStackInternal` · seen in 2/3 runs · NEW HookRigOrchestrator._configurePoolStackInternal does not verify that getHookPermissions supports every enabled block, so enabling a return-delta block on ProTradingPoolPresetHook reverts every swap.	FIXED	registerPoolStack calls _validateBitmapCompatibility -> validateBitmapAndPermissions (AccessControlBlock.sol:41-49), which reverts unless the hook address flags cover permissionsForBitmap(bitmap).
S2-F36	96	**One-wei sell between buys resets transient wallet cap balance to zero** `HookRigOrchestrator._runAfterSwapPipeline` · seen in 3/3 runs · NEW HookRigOrchestrator._runAfterSwapPipeline sets the transient wallet balance to zero on any sell, so a buyer who interleaves one-wei sells in unlock bypasses maxWalletBps.	FIXED	WalletCapBlock.sol:94-100: a sell now lowers the expected balance by `sold` (prevExpected - sold) instead of setting it to zero.
S2-F37	96	**Tax update adds 1000 bps for ArbRecapture and blocks lowering taxes** `HookRigOrchestrator.setBuySellTax` · seen in 3/3 runs · NEW HookRigOrchestrator.setBuySellTax adds 1000 bps for BIT_ARB_RECAPTURE while registerPoolStack omits it, so setBuySellTax reverts when poolAdmin tries to lower taxes.	FIXED	setBuySellTax and registration share BuySellTaxLogic.validateTaxBudget (BuySellTaxBlock.sol:143-169); there is no ArbRecapture 1000-bps addition.
S2-F38	96	**Waterfall skips _processKingOfThePoolBuy on exact-output buy swaps** `KingOfThePoolBlock._processKingOfThePoolBuy` · seen in 3/3 runs · NEW The after-swap waterfall skips _processKingOfThePoolBuy when the unspecified token is the quote token, so a buyer who uses an exact-output swap pays zero fee.	FIXED	AfterSwapDeltaAggregator.sol:61-66 reverts ExactOutputBuyNotSupported for exact-output buys when King or AutoBurn is enabled.
S2-F39	96	**Unbounded minBuyAmountForReset collects jackpot fees without crowning a king** `KingOfThePoolBlock.initKing` · seen in 2/3 runs · NEW KingOfThePoolLogic.initKing places no upper bound on minBuyAmountForReset, so an unreachable threshold collects buy fees while currentKing stays zero forever.	BY-DESIGN	initKing caps minBuyAmountForReset at MAX_MIN_BUY_AMOUNT_FOR_RESET = 1e9 ether; creator parameter sizing for low-supply tokens is documented in KingOfThePoolBlock NatSpec.

ID	Conf.	Finding	Status	Current evidence
S2-F40	96	**Exact-output buys skip KingOfThePoolBlock and avoid the jackpot fee** `KingOfThePoolBlock.processBuy` · seen in 3/3 runs · KNOWN (2 scans) AfterSwapDeltaAggregator skips KingOfThePoolLogic.processBuy when unspecifiedCurrency is quoteCurrency, so any buyer who uses an exact-output swap pays zero jackpot fee.	**FIXED**	Exact-output buys revert with ExactOutputBuyNotSupported (AfterSwapDeltaAggregator.sol:61-66), so the jackpot fee can no longer be skipped this way.
S2-F41	96	**Factory transfers full unvested supply to NativeTickLaunch and locks excess** `LaunchpadFactory.launchNativeTick` · seen in 3/3 runs · NEW LaunchpadFactory.launchNativeTick transfers all available tokens to NativeTickLaunch even when the liquidity bands use less, so the unseeded tokens stay locked forever.	**FIXED**	LaunchpadFactory.launchNativeTick validates all bands (ticks, spacing, single-sided placement, positive liquidityDelta, totalBandTokens <= availableForCurve) and NativeTickLaunch burns any unseeded residual to 0xdEaD.
S2-F42	96	**Preset hooks passed to launchVirtualXYK lack beforeInitialize protection** `LaunchpadFactory.launchVirtualXYK` · seen in 2/3 runs · KNOWN (2 scans) LaunchpadFactory.launchVirtualXYK accepts preset hooks whose permissions disable beforeInitialize, so an attacker can initialize the pool early and block graduation.	**FIXED**	_resolveHook rejects any hook other than defaultOrchestrator (LaunchpadFactory.sol:192-196), so preset hooks cannot be passed.
S2-F43	96	**Fee-on-transfer limit order cancellation drains shared hook token reserves** `LimitOrdersBlock.placeLimitOrder` · seen in 2/3 runs · NEW LimitOrdersLogic.placeOrder records inputAmount without measuring received tokens, so a maker who cancels a fee-on-transfer order withdraws other users' tokens.	**FIXED**	placeOrder checks balance delta: `token.balanceOf(this) - balBefore != inputAmount` reverts (LimitOrdersBlock.sol:198-200).
S2-F44	96	**Unguarded executeFillSwap lets any caller sweep executor balances** `LimitOrderSwapExecutor.executeFillSwap` · seen in 2/3 runs · NEW LimitOrderSwapExecutor.executeFillSwap has no caller check and sends any remaining token or ETH balance to recipient, so any user can sweep funds held by the executor.	**FIXED**	executeFillSwap: `if (msg.sender != hook) revert OnlyHookCaller` (LimitOrdersBlock.sol:51).
S2-F45	96	**Updating managed fee before pool initialization reverts in PoolManager** `ManagedFeeBlock.setManagedPoolFee` · seen in 2/3 runs · NEW ManagedFeeBlock.setManagedPoolFee calls poolManager.updateDynamicLPFee unconditionally, so the fee manager cannot update the staged fee before the pool initializes.	**FIXED**	ManagedFeeBlock.setManagedPoolFee checks StateLibrary.getSlot0 sqrtPriceX96 != 0 and stages the fee without calling PoolManager when uninitialized (ManagedFeeBlock.sol:70-82). Test test_Fix_ManagedFee_StagingBeforePoolInit.
S2-F46	96	**Unallocated and rounding-leftover curve tokens stay locked in NativeTickLaunch** `NativeTickLaunch.unlockCallback` · seen in 3/3 runs · KNOWN (2 scans) NativeTickLaunch.unlockCallback settles only the band liquidity deltas and has no sweep path, so all unallocated tokens stay locked in the contract forever.	**FIXED**	NativeTickLaunch.unlockCallback sweeps any unseeded residual project tokens to DEAD_ADDRESS (0xdEaD) after settling bands (NativeTickLaunch.sol:102-108).
S2-F47	96	**Attacker binds referrer to untrusted router and siphons swap fees** `ReferralSplitBlock.registerReferrer` · seen in 2/3 runs · NEW ReferralSplitBlock.registerReferrer permanently sets traderReferrer for msg.sender, so an attacker who calls it through an untrusted router takes 3% of swaps through that router.	**FIXED**	ReferralSplitBlock.registerReferrer rejects trusted routers and hook self-registration (_isDisallowedReferrer), and untrusted routers receive attributedTrader = address(0) (ReferralSplitBlock.sol:92-104).

ID	Conf.	Finding	Status	Current evidence
S2-F48	96	**Stepped curve graduation truncates sqrtPrice via 32-bit shift** `SteppedTrancheLaunch._graduateToV4` · seen in 2/3 runs · NEW SteppedTrancheLaunch._graduateToV4 shifts Math.sqrt(ratioX128) left by 32 bits when tokensToSeed exceeds uint64.max, which truncates the initial pool price.	FIXED	Tiered X192/X160(<<16)/X128(<<32) sqrt computation (SteppedTrancheLaunch.sol:237-246) limits truncation to negligible precision.
S2-F49	96	**Stepped curve graduation donates leftover tokens to locked LP position** `SteppedTrancheLaunch.unlockCallback` · seen in 2/3 runs · NEW SteppedTrancheLaunch.unlockCallback passes leftover project tokens to poolManager.donate on its own locked LP position, so nobody can ever claim or trade them.	BY-DESIGN	Leftover donation to the locked graduation LP is documented as an accepted design limit (docs/04:91; docs/07 TODO-21 "Accepted").
S2-F50	96	**Unbounded loop in executePending reverts swaps when dust orders accumulate** `TWAMMBlock.executePending` · seen in 2/3 runs · NEW An attacker submits many one-year dust orders in one timestamp so the unbounded while loop in TWAMMLogic.executePending runs out of gas on later blocks.	FIXED	submitOrder enforces MAX_ACTIVE_TWAMM_ORDERS=256 and MIN_TWAMM_ORDER_AMOUNT=1e15 (TWAMMBlock.sol:106-107,200-212), preventing dust-order loop exhaustion.
S2-F51	96	**Fee-on-transfer TWAMM order cancellation drains shared hook token reserves** `TWAMMBlock.submitTWAMMOrder` · seen in 2/3 runs · NEW TWAMMLogic.submitOrder records totalAmount without measuring received tokens, so a caller who cancels a fee-on-transfer order in the same block takes other users' tokens.	FIXED	submitOrder verifies exact ERC-20 balance delta received (balAfter - balBefore != totalAmount reverts) and tracks per-pool escrow (TWAMMBlock.sol:214-224).
S2-F52	96	**Unupdated invariantK after rounded buy lets seller drain prior buyer quote** `VirtualXYKLaunch.sell` · seen in 3/3 runs · KNOWN (2 scans) VirtualXYKLaunch leaves invariantK unchanged when upward division on buy increases the reserve product, so a seller extracts prior buyers' excess quote in sell.	BY-DESIGN	VirtualXYKLaunch.buy rejects tokensOut == 0 (ZeroAmount) and preserves constant invariantK required by invariant properties GL-12 and SP-12 (VirtualXYKLaunch.sol:113).
S2-F53	96	**Graduation donates leftover tokens to an unclaimable LP fee account** `VirtualXYKLaunch.unlockCallback` · seen in 3/3 runs · NEW VirtualXYKLaunch.unlockCallback passes leftover project tokens to poolManager.donate on a locked position with no fee-collection function, so the contract freezes those tokens forever.	BY-DESIGN	unlockCallback donates leftovers to the locked LP (VirtualXYKLaunch.sol:252-264), an accepted limit per docs/04:91 and docs/07 TODO-21.
S2-F54	94	**Reward points use raw unspecified token units without normalization** `SwapRewardsBlock._accrueSwapRewards` · seen in 3/3 runs · NEW SwapRewardsBlock._accrueSwapRewards computes points from the raw unspecified token amount, so a trader who selects the project token as the unspecified leg inflates points.	FIXED	HookRigOrchestrator.sol:409-412 computes points from project-token volume whichever leg is unspecified.
S2-F55	92	**Tax update rejects zero recipient sentinel before checking stored recipient** `BuySellTaxBlock._updateBuySellTax` · seen in 3/3 runs · NEW BuySellTaxLogic.update rejects a zero newRecipient before it checks the stored taxRecipient, so the poolAdmin cannot use zero to preserve the existing recipient.	FIXED	BuySellTaxLogic.update resolves effectiveRecipient = newRecipient != address(0) ? newRecipient : cfg.taxRecipient before checking non-zero tax (BuySellTaxBlock.sol:106-116). Test test_Block09_BuySellTax_ClaimsAndBounds.

ID	Conf.	Finding	Status	Current evidence
S2-F56	92	**Preset hooks allow registering extra blocks missing from hook permissions** `FairLaunchPresetHook.getHookPermissions` · seen in 2/3 runs · NEW Preset hook contracts override getHookPermissions with fixed preset flags but accept any Tier 3 bitmap in registerPoolStack, so unsupported return-delta blocks revert every swap.	**FIXED**	validateBitmapAndPermissions requires the hook address flags to cover permissionsForBitmap(bitmap) (AccessControlBlock.sol:45-48), called in registerPoolStack:100.
S2-F57	92	**Premature graduation call or curve LP seeding corrupts post-graduation timers** `HookRigOrchestrator.notifyPoolGraduated` · seen in 1/3 runs · KNOWN (2 scans) HookRigOrchestrator.notifyPoolGraduated allows launchpadFactory to reset clocks before graduation and records the curve's full-range LP block in JITGuard's shared rangeKey slot.	**FIXED**	resetGraduationClocks allows one call only (AntiSnipeBlock.sol:82 PoolAlreadyGraduated); curves call it right after initialize; _beforeAddLiquidity skips _recordLPAddition for the curve (Orchestrator:273).
S2-F58	92	**Unbounded anti-snipe surcharge and duration divert buyer deposits to creator** `SteppedTrancheLaunch.initializeCurve` · seen in 1/3 runs · KNOWN (2 scans) SteppedTrancheLaunch.initializeCurve accepts antiSnipeSurchargeBps up to 9999 and unlimited antiSnipeBlocks, so a creator can divert 99.99% of buyer quote payments into creator claims.	**FIXED**	SteppedTrancheLaunch enforces MAX_ANTI_SNIPE_BLOCKS = 50_400 and claimStalledSurchargeRefund refunds pro-rata buyerSurcharges if a launch stalls (SteppedTrancheLaunch.sol :28,76,245-272).
S2-F59	92	**Unbounded active TWAMM order loop runs out of gas during swaps** `TWAMMBlock._executePendingTWAMMOrders` · seen in 2/3 runs · NEW TWAMMBlock._executePendingTWAMMOrders iterates through every active order on each swap and liquidity call, so an attacker who submits many dust orders blocks the pool.	**FIXED**	Active TWAMM orders are capped at MAX_ACTIVE_TWAMM_ORDERS=256 and require totalAmount >= 1e15 (TWAMMBlock.sol:106-107,203).
S2-F60	85	**Pre-initialization TWAMM order timestamp overwrite inflates pool sell rate forever** `TWAMMBlock._initTWAMM` · seen in 1/3 runs · NEW Submitting a TWAMM order before pool initialization causes _initTWAMM to overwrite lastVirtualOrderTimestamp, so executePending drops the expired order without subtracting sellRatePerSecond.	**FIXED**	submitOrder requires state.lastVirtualOrderTimestamp != 0 (pool initialized via _initTWAMM) and tracks order.rateRemoved so expired rates are subtracted once (TWAMMBlock.sol:201,404-411).
S2-F61	82	**Zero managed fee cap is replaced with five percent cap** `ManagedFeeBlock.stage` · seen in 1/3 runs · NEW ManagedFeeLogic.stage replaces a zero feeCap with MAX_MANAGED_FEE_CAP, so the fee manager can raise the pool fee to five percent despite a zero cap.	**FIXED**	ManagedFeeLogic.stage uses params.feeCap as-is (ManagedFeeBlock.sol:32); a zero cap stays zero and is no longer replaced with MAX_MANAGED_FEE_CAP.
S2-F62	80	**Empty or sub-unit native tick bands graduate pool with zero liquidity** `NativeTickLaunch.initializeAndSeedNativeTicks` · seen in 1/3 runs · NEW NativeTickLaunch.initializeAndSeedNativeTicks allows empty bands or bands whose liquidity rounds to zero, which graduates the pool with zero liquidity and locks all tokens.	**FIXED**	LaunchpadFactory.launchNativeTick computes liquidityDelta for every band and reverts InvalidCurveConfiguration if any band has zero liquidity or invalid ticks (LaunchpadFactory.sol:308-355).
S2-F63	78	**Standard pools allow any caller to initialize at an extreme price** `HookRigOrchestrator._beforeInitialize` · seen in 1/3 runs · NEW HookRigOrchestrator._beforeInitialize checks the caller only for launchpad pools, so an attacker can initialize a registered standard pool at an extreme tick.	**FIXED**	HookRigOrchestrator._verifyPoolInit calls AccessControlLogic.verifyInitializer in both _beforeInitialize and _afterInitialize, restricting standard pool init to poolAdmin or owner. Test test_Fix_SetLaunchpadFactory_OneTimeLockAndStandardPoolInitAuth.

ID	Conf.	Finding	Status	Current evidence
S2-F64	78	**Swaps through untrusted routers crown the router and lock the jackpot** `HookRigOrchestrator.claimKingPot` · seen in 1/3 runs · NEW Swapping through an untrusted router sets round.winner to the router contract address, so the buyer cannot call claimKingPot and the jackpot stays locked.	**FIXED**	AfterSwapDeltaAggregator passes attributedTrader = traderIsSender ? address(0) : ctx.trader to KingOfThePool, so untrusted routers contribute to active pots without being crowned (AfterSwapDeltaAggregator.sol:71,158-170).
S2-F65	78	**Untrusted router crowned as king cannot call validateAndMarkClaimed** `KingOfThePoolBlock.validateAndMarkClaimed` · seen in 1/3 runs · NEW KingOfThePoolLogic.validateAndMarkClaimed requires msg.sender to equal round.winner, so jackpots won through an untrusted router contract can never be claimed.	**FIXED**	Untrusted routers pass buyer = address(0) to processWaterfallKing and can never be crowned King (AfterSwapDeltaAggregator.sol:71,158-170). Test test_Fix_UntrustedRouterGuards_WalletCap_King_Referral.
S2-F66	75	**Cancelling a fee-on-transfer limit order withdraws full nominal input amount** `LimitOrdersBlock.cancelLimitOrder` · seen in 1/3 runs · NEW LimitOrdersBlock.cancelLimitOrder transfers the unadjusted order.inputAmount from the shared hook balance, so cancelling a fee-on-transfer order drains other makers' escrowed tokens.	**FIXED**	Exact-received check in placeOrder (L198-200) plus per-pool escrow guard InsufficientPoolEscrow in cancelOrder (L240-244).
S2-L01	lead	**JIT exit penalty configuration is stored but never read on removal** `JITGuardBlock._initJITGuard` Code smells: `JITGuardBlock._initJITGuard` validates `jitExitPenaltyBps <= MAX_JIT_PENALTY_BPS` and writes `_jitConfigs[poolId].jitExitPenaltyBps` (lines 1689-1695), but no function in `JITGuardBlock` or `HookRigOrchestrator` ever reads `jitExitPenaltyBps`. — JITGuardBlock stores jitExitPenaltyBps in _jitConfigs during initialization, but no function reads that value when liqui…	**FIXED**	jitExitPenaltyBps enforces a minimum 1-block hold floor in enforceOnRemove when minLPHoldBlocks == 0, and docs/04 documents beforeRemoveLiquidity hold semantics (JITGuardBlock.sol:124).
S2-L02	lead	**Shared native ETH limit order escrow lacks per-pool balance isolation** `LimitOrdersBlock.placeLimitOrder` Code smells: `LimitOrdersLogic.placeOrder` (lines 2123-2129) holds native ETH and ERC-20 limit order escrows in the shared `HookRigOrchestrator` balance without checking `_poolMeta[poolId].configured`. — LimitOrdersBlock stores native ETH escrows in the shared hook balance, though draining native ETH requires another path that under-deposits ETH.	**FIXED**	Per-pool escrow `limitOrderPoolEscrow[poolId][currency]` checked on cancel/claim (L240-244,286-290). Native ETH requires msg.value==inputAmount (L194).

7. Scan 3 — 2026-10-15 07:18 (ede9c74)

1-pass scan of commit `ede9c74` (pass-3 findings). Remediation: b0859d2 (feat: audit pass 3 and fuzz testing). 69 findings and 6 leads; current status: 63 fixed, 12 by-design.

ID	Conf.	Finding	Status	Current evidence
S3-F01	98	**Exact-output buy sends buyer quote currency to the dead address** `AfterSwapDeltaAggregator._executeAfterSwapDeltaWaterfall` On an exact-output buy, _executeAfterSwapDeltaWaterfall passes quoteCurrency to AutoBurnLogic, so the hook sends quote tokens to the dead address.	**FIXED**	An exact-output buy reverts before the burn. AutoBurn is gated by `ctx.unspecifiedCurrency == ctx.projectCurrency` (AfterSwapDeltaAggregator.sol:113-116).
S3-F02	98	**Preset hooks disable beforeInitialize and let callers block graduation** `HookPermissionMasks.permissionsForBitmap` permissionsForBitmap disables beforeInitialize when BIT_TWAMM is off, so an attacker can initialize a preset launchpad pool early and block graduation forever.	**FIXED**	_afterInitialize (enabled whenever bitmap!=0) reverts if sender isn't authorizedCurve/launchpadFactory (HookRigOrchestrator.sol:221-223), blocking early init.
S3-F03	98	**Quote currency surcharge seed is diverted to poolAdmin in seedKingPotFromLaunchCurve** `KingOfThePoolBlock.seedKingPotFromLaunchCurve` KingOfThePoolLogic.seedPotFromLaunchCurve requires the project token while SteppedTrancheLaunch sends the quote token, so the pool admin takes the Round 1 pot.	**FIXED**	seedPotFromLaunchCurve (KingOfThePoolBlock.sol:264-271) accepts any currency into an empty pot and reverts otherwise. No poolAdmin credit path remains.
S3-F04	98	**Currency check in seedPotFromLaunchCurve sends launchpad jackpot seed to poolAdmin** `KingOfThePoolBlock.seedPotFromLaunchCurve` The seedPotFromLaunchCurve function requires projectCurrency while SteppedTrancheLaunch sends currency0, so the contract sends the Round 1 jackpot surcharge to poolAdmin.	**FIXED**	The quote currency0 seed from SteppedTrancheLaunch.sol:260 is accepted when the pot is empty (KingOfThePoolBlock.sol:266-271). There is no poolAdmin diversion.
S3-F05	98	**KingOfThePoolLogic.seedPotFromLaunchCurve rejects quoteCurrency and credits poolAdmin** `KingOfThePoolLogic.seedPotFromLaunchCurve` The poolAdmin receives all graduation surcharge funds from SteppedTrancheLaunch because seedPotFromLaunchCurve requires projectCurrency while _graduateToV4 passes quoteCurrency.	**FIXED**	seedPotFromLaunchCurve no longer requires projectCurrency (only empty pot or matching currency) and reverts KingPotSeedNotAccepted instead of crediting poolAdmin.
S3-F06	98	**SteppedTrancheLaunch._graduateToV4 passes currency0 to seedKingPotFromLaunchCurve and diverts pot** `SteppedTrancheLaunch._graduateToV4` SteppedTrancheLaunch._graduateToV4 passes poolKey.currency0 to seedKingPotFromLaunchCurve, which checks currency == meta.projectCurrency, so the contract sends the jackpot seed to poolAdmin.	**FIXED**	seedPotFromLaunchCurve now accepts the curve's currency0 and credits a live pot, else reverts. It never credits poolAdmin (KingOfThePoolBlock.sol:264-271).
S3-F07	98	**TWAMM order cancellation withholds streamed tokens with no claim path** `TWAMMBlock.cancelTWAMMOrder` A user who submits a TWAMM order loses all streamed tokens forever, because TWAMMBlock deducts streamed amounts on cancel and has no claim function.	**FIXED**	cancelTWAMMOrder settles accrued virtual volume, transfers claimableOutput to maker, and refunds all unswapped input (TWAMMBlock.sol:264-317).
S3-F08	98	**TWAMMLogic.executePending retires orders without a swap or claim credit** `TWAMMBlock.executePending` TWAMMBlock records virtual volume and deletes expired orders without a swap or claim mechanism, so the contract locks user tokens forever.	**FIXED**	executePendingWithSwap swaps streamed volume via PoolManager/TWAMMSwapExecutor and credits pro-rata claimableOutput claimable via claimTWAMMOrder (TWAMMBlock.sol:432-550).

ID	Conf.	Finding	Status	Current evidence
S3-F09	98	**Streamed TWAMM order deposits are never swapped and stay locked forever** `TWAMMBlock.submitTWAMMOrder` The contract holds tokens from submitTWAMMOrder and never swaps or pays out the streamed share, so every order loses its streamed tokens forever.	**FIXED**	Streamed TWAMM deposits are swapped in _beforeSwap and via TWAMMSwapExecutor, with pro-rata output paid via claimTWAMMOrder or cancelTWAMMOrder (TWAMMBlock.sol).
S3-F10	95	**Exact-output buy skips the KingOfThePool fee check** `AfterSwapDeltaAggregator._executeAfterSwapDeltaWaterfall` AfterSwapDeltaAggregator skips processWaterfallKing when unspecifiedCurrency is the quote token, so a buyer who uses an exact-output swap pays zero pot fee.	**FIXED**	An exact-output buy on a King-enabled pool reverts ExactOutputBuyNotSupported, so the pot fee cannot be skipped (AfterSwapDeltaAggregator.sol:60-65).
S3-F11	95	**Zero rounded arbitrage cut still advances referenceTick to postSwapTick** `ArbRecaptureBlock._computeArbRecaptureCutBps` ArbRecaptureLogic.computeCutBps updates referenceTick to postSwapTick even when rawCutBps rounds to zero, so an arbitrageur splits a swap and pays zero fee.	**FIXED**	computeCutBps only sets `state.referenceTick = postSwapTick` when `rawCutBps > 0` (ArbRecaptureBlock.sol:69-71).
S3-F12	95	**Split swaps reset referenceTick in computeCutBps with zero fee** `ArbRecaptureBlock.computeCutBps` ArbRecaptureLogic.computeCutBps updates referenceTick when rawCutBps rounds to zero, so a trader splits one large swap into small steps and pays zero fee.	**FIXED**	referenceTick advances only when rawCutBps>0, and the tick move is measured from referenceTick, so split swaps add up their moves.
S3-F13	95	**One-excess-tick swaps advance referenceTick in ArbRecaptureLogic with zero cut** `ArbRecaptureLogic.computeCutBps` An arbitrageur can split a swap into steps of arbTriggerTickDelta plus one tick to reset referenceTick and pay zero recapture fee.	**FIXED**	A zero-cut swap no longer advances referenceTick (`if (rawCutBps > 0)`), and refTickMoved adds up the steps since the last charged swap.
S3-F14	95	**Inverted tick check triggers buyback above the floor price** `AutoBuybackBlock._processAutoBuyback` accumulateAndCheckTrigger inverts the tick comparison for both token orders, so the contract deploys buyback reserves only above the floor price.	**FIXED**	AutoBuybackBlock.sol:84-86 now triggers when postSwapTick <= triggerFloorTick (token0 project) or postSwapTick >= triggerFloorTick (token1 project). Test test_Fix_AutoBuyback_FloorTriggerBothTokenOrders.
S3-F15	95	**Shared router key in _enforceJITGuardOnRemove lets attackers block LP withdrawals** `JITGuardBlock._enforceJITGuardOnRemove` JITGuardLogic.recordAddition writes to the shared router key, so an attacker who adds one wei of liquidity blocks withdrawals for the position owner.	**FIXED**	JITGuardLogic.recordAddition applies a 5% anti-griefing threshold on shared untrusted-router keys (addedLiq * 20 >= trackedLiq) so 1-wei adds cannot re-lock existing LPs (JITGuardBlock.sol:34-48).
S3-F16	95	**Shared router and salt overwrite lpAddedBlock in _recordLiquidityAddition** `JITGuardBlock._recordLiquidityAddition` recordAddition overwrites lpAddedBlock for any shared sender and salt, so an attacker can add one wei of liquidity to reset a victim cooldown.	**FIXED**	1-wei adds below 5% of tracked liquidity on a shared untrusted router do not overwrite lpAddedBlock (JITGuardBlock.sol:34-48).
S3-F17	95	**Router senderKey write in _recordLPAddition locks other router LPs** `JITGuardBlock._recordLPAddition` JITGuardLogic.recordAddition overwrites the shared senderKey block timestamp, so an attacker who adds one wei via the same router locks another LP position.	**FIXED**	Trusted-router positions track per-owner lpTrackedLiquidity[ownerKey] and shared untrusted routers ignore dust (<5%) timer resets (JITGuardBlock.sol:34-155).

ID	Conf.	Finding	Status	Current evidence
S3-F18	95	**Router cooldown lookup in enforceOnRemove blocks LP withdrawals** `JITGuardBlock.enforceOnRemove` The recordAddition function stores the cooldown under the router address, so an attacker who adds one wei of liquidity blocks withdrawals for other users.	**FIXED**	enforceOnRemove uses per-owner lpTrackedLiquidity[ownerKey] when sufficient and applies the 5% anti-griefing filter on shared routers (JITGuardBlock.sol:72-155).
S3-F19	95	**One-wei deposit via a shared router resets all LP hold timers** `JITGuardBlock.recordAddition` JITGuardLogic.recordAddition overwrites the shared router timestamp on every deposit, so any depositor resets the withdrawal hold period for other router users.	**FIXED**	Dust deposits (<5% of tracked liquidity) on shared untrusted routers no longer reset lpAddedBlock (JITGuardBlock.sol:34-48).
S3-F20	95	**Shared senderKey in JITGuardLogic.recordAddition lets attackers freeze LP removals** `JITGuardLogic.recordAddition` An attacker who adds one wei of liquidity through a shared router resets senderKey and blocks liquidity removal for other users.	**FIXED**	Shared untrusted-router keys require addedLiq >= 5% of tracked liquidity to advance lpAddedBlock (JITGuardBlock.sol:34-48).
S3-F21	95	**Creator vesting starts before graduation and drains buyer quote deposits** `LaunchpadFactory._deployTokenAndVest` LaunchpadFactory._deployTokenAndVest unlocks creator tokens before graduation, so the creator can sell those tokens into the curve and take buyer quote tokens.	**FIXED**	LaunchpadVestingWallet reverts CurveNotGraduatedYet on release() while the bonding curve is active (LaunchpadFactory.sol:29-66).
S3-F22	95	**Single-tranche graduation cap forces SteppedTrancheLaunch to graduate in tranche zero** `LaunchpadFactory.launchSteppedTranche` LaunchpadFactory caps graduationQuoteTarget at the quote value of one tranche, so SteppedTrancheLaunch always graduates in tranche zero and skips higher tranches.	**BY-DESIGN**	Single-tranche graduation target bound in LaunchpadFactory is intentional and pairs with per-tranche marginal pricing (docs/05).
S3-F23	95	**Hardcoded false in _swapFilledOrderOnClaim refunds unswapped input tokens** `LimitOrdersBlock._swapFilledOrderOnClaim` LimitOrdersBlock._swapFilledOrderOnClaim returns false in production hooks, so claimLimitOrder returns the original input token without a swap.	**FIXED**	_swapFilledOrderOnClaim returns true (LimitOrdersBlock.sol:400-402); no override in HookRigOrchestrator.
S3-F24	95	**Filled limit orders never swap and refund original input tokens on claim** `LimitOrdersBlock.claimLimitOrder` LimitOrdersBlock._swapFilledOrderOnClaim returns false in HookRigOrchestrator, so LimitOrdersLogic.claimOrder returns the original input tokens to the maker instead of output tokens.	**FIXED**	claimOrder calls _executeClaimSwap via executor when swapOnClaim, paying output tokens (LimitOrdersBlock.sol:294-295,319-327).
S3-F25	95	**One-wei unclaimed order in a crossed bucket blocks placeLimitOrder forever** `LimitOrdersBlock.placeLimitOrder` An attacker who places a one-wei order and never claims it after fill keeps bucket.totalInputAmount above zero, which blocks placeLimitOrder at that tick forever.	**FIXED**	placeOrder resets bucket.filled when price is back on the unfilled side (LimitOrdersBlock.sol:179-190). Reverts only while target is still crossed.
S3-F26	95	**Unclaimed dust in a crossed bucket makes placeOrder revert forever** `LimitOrdersBlock.placeOrder` An attacker leaves a tiny unclaimed order in a crossed bucket, and placeOrder rejects all new orders at that tick forever.	**FIXED**	Same placeOrder reset (LimitOrdersBlock.sol:183-188). Dust only blocks while price stays past target, when a new order would be marketable anyway.

ID	Conf.	Finding	Status	Current evidence
S3-F27	95	**Unclaimed one-wei order locks LimitOrdersLogic.placeOrder at that tick forever** `LimitOrdersLogic.placeOrder` An attacker leaves a one-wei filled order unclaimed at a tick, so LimitOrdersLogic.placeOrder reverts with BucketAlreadyFilled for every caller at that tick forever.	**FIXED**	BucketAlreadyFilled is now raised only if currentTick is still past targetTick; otherwise filled is cleared (LimitOrdersBlock.sol:183-188).
S3-F28	95	**Floor rounding at the allocation cap in SteppedTrancheLaunch.buy blocks graduation** `SteppedTrancheLaunch.buy` SteppedTrancheLaunch.buy rounds netQuoteForTokens down when a buy exceeds remainingTokens, so the last dust balance returns zero tokens and blocks graduation for every buyer.	**FIXED**	In the cap branch, tokensOut=remainingTokens and netQuoteForTokens is recomputed with Math.Rounding.Ceil (SteppedTrancheLaunch.sol:132-137).
S3-F29	95	**Single marginal price lookup in SteppedTrancheLaunch.buy skips higher tranche steps** `SteppedTrancheLaunch.buy` SteppedTrancheLaunch.buy prices the whole order at currentMarginalPriceQ96, so a buyer in tranche zero buys the entire allocation at basePriceQ96 and skips all price steps.	**BY-DESIGN**	Single marginal price lookup is bounded by the factory's single-tranche graduationQuoteTarget cap (LaunchpadFactory.sol, docs/05).
S3-F30	95	**Live balanceOf snapshot in _enforceVestingSellCap allows flash-loan sell cap bypass** `VestingSellCapBlock._enforceVestingSellCap` enforceSellCap reads the live balance of the seller during _afterSwap, so a seller can borrow pool tokens before the swap and sell their full balance.	**BY-DESIGN**	PoolManager flash-takes are netted via currencyDelta and outflows lower the epoch snapshot; external ERC-20 flash-loan limits are documented in VestingSellCapBlock and docs/04 §3.7.
S3-F31	95	**Pool token borrow before swap inflates startOfEpochBalance in enforceSellCap** `VestingSellCapBlock.enforceSellCap` A seller who flash-takes projectCurrency from poolManager before the first daily swap inflates ep.startOfEpochBalance and sells their entire balance in one transaction.	**FIXED**	_preSellBalance adds TransientStateLibrary.currencyDelta for the locker (VestingSellCapBlock.sol:96-97), so a flash take from poolManager is subtracted.
S3-F32	95	**Zero post-swap seller balance in enforceSellCap reverts every sell above one wei** `VestingSellCapBlock.enforceSellCap` When currentBal is zero, VestingSellCapLogic.enforceSellCap sets ep.startOfEpochBalance to tokensSold, so newSoldTotal > maxAllowed reverts every sell above one wei.	**FIXED**	VestingSellCapBlock.sol:56 `if (preSellBal == 0) return;` skips the cap instead of setting startOfEpochBalance=tokensSold.
S3-F33	95	**Setting startOfEpochBalance to tokensSold in VestingSellCapLogic.enforceSellCap reverts sells** `VestingSellCapLogic.enforceSellCap` Every sell through an untrusted router reverts with DailyVestingSellCapExceeded because enforceSellCap sets startOfEpochBalance to tokensSold and requires a smaller sale.	**FIXED**	VestingSellCapLogic._preSellBalance returns 0 for stateless untrusted routers (sellerIsSwapper && postSettlementBalance == 0), preventing false reverts on pre-settled router sells (VestingSellCapBlock.sol:98-105).
S3-F34	95	**Untrusted router contract bypasses maxWalletBps in _enforceWalletCapAfterSwap** `WalletCapBlock._enforceWalletCapAfterSwap` WalletCapLogic.enforceAfterBuy checks the token balance of an untrusted router instead of the recipient, so a buyer who uses a custom router bypasses maxWalletBps.	**FIXED**	WalletCapLogic.enforceAfterSwap reverts UntrustedRouterNotAllowedForWalletCap when traderIsSender is true during active WalletCap buys (WalletCapBlock.sol:55-68). Test test_Fix_UntrustedRouterGuards_WalletCap_King_Referral.
S3-F35	95	**Untrusted router contract bypasses WalletCapBlock.enforceAfterSwap** `WalletCapBlock.enforceAfterSwap` The enforceAfterBuy function checks the token balance of trader, so a buyer who uses an untrusted router contract can buy the entire token supply.	**FIXED**	HookRigOrchestrator passes trader == sender to _enforceWalletCapAfterSwap, which reverts UntrustedRouterNotAllowedForWalletCap on untrusted routers (WalletCapBlock.sol:55-68).

ID	Conf.	Finding	Status	Current evidence
S3-F36	95	**Zero router token balance in WalletCapLogic.enforceAfterBuy lets buyers exceed maxWalletBps** `WalletCapLogic.enforceAfterBuy` A buyer who swaps through an untrusted router contract makes AccessControlLogic.resolveUser return the router address, so WalletCapLogic.enforceAfterBuy checks the zero router balance.	**FIXED**	Untrusted routers (trader == sender) are rejected on buys while WalletCap is active (UntrustedRouterNotAllowedForWalletCap, WalletCapBlock.sol:55-68).
S3-F37	90	**HookData fallback in _resolveUser lets callers spoof allowlisted users** `AccessControlBlock._resolveUser` AccessControlLogic.resolveUser reads the user address from hookData when msgSender fails, so an attacker can impersonate an allowlisted user.	**FIXED**	AccessControlBlock.sol:238 resolveUser ignores hookData (unnamed param); trusted router returns msgSender() or routerSender, untrusted returns routerSender.
S3-F38	90	**Unauthenticated hookData fallback in resolveUser allows user impersonation** `AccessControlBlock.resolveUser` The resolveUser function reads the user address from caller hookData when msgSender reverts, so an attacker can impersonate any allowlisted address.	**FIXED**	resolveUser catch branch returns routerSender, not a hookData-decoded address (AccessControlBlock.sol:242-246).
S3-F39	90	**Trusted router catch block decodes unverified victim address from hookData** `AccessControlLogic.resolveUser` An attacker passes a victim address in hookData when IMsgSender.msgSender reverts, which sets startOfEpochBalance and lastSwapTimestamp for the victim.	**FIXED**	AccessControlLogic.resolveUser catch block now `return routerSender;`, so no victim address is decoded from hookData.
S3-F40	90	**Missing hook permission check in _validateBitmapCompatibility causes swaps to revert** `HookRigOrchestrator._validateBitmapCompatibility` HookRigOrchestrator.registerPoolStack does not check getHookPermissions against the bitmap, so return-delta blocks on ProTradingPoolPresetHook make every swap revert.	**FIXED**	_validateBitmapCompatibility (Orchestrator:90-92) checks the bitmap against the hook-address permission flags through validateBitmapAndPermissions (AccessControlBlock.sol:45-48).
S3-F41	90	**Anyone can front-run registerPoolStack to hijack poolAdmin or block launch** `HookRigOrchestrator.registerPoolStack` registerPoolStack checks no caller permission when authorizedCurve is zero, so an attacker can front-run token deployment and lock the PoolId configuration forever.	**FIXED**	Undeployed token registration is blocked (TokenNotDeployed) and registered standard pools restrict initialize to poolAdmin/owner (AccessControlBlock.sol:49-105).
S3-F42	90	**Preset hooks accept bitmaps that exceed their static hook permissions** `HookRigPresets.registerPoolStack` Preset hook contracts do not verify that getHookPermissions supports enabledBlocksBitmap, so a pool registered with a return-delta block on ProTradingPoolPresetHook fails every swap.	**FIXED**	Presets inherit HookRigOrchestrator.registerPoolStack -> _validateBitmapCompatibility, which checks the preset hook address flags (AccessControlBlock.sol:41-49).
S3-F43	90	**Buyer becomes currentKing with zero fee when waterfall cap is full** `KingOfThePoolBlock._processKingOfThePoolBuy` A buyer whose swap reaches the 2500 bps cap before KingOfThePool pays zero pot fee, yet processBuy still crowns that buyer as currentKing.	**BY-DESIGN**	The code comment at KingOfThePoolBlock.sol:151-153 explicitly crowns non-dust buys even when the 25% cap is already full and potContribution==0.
S3-F44	90	**Zero potContribution in processBuy still crowns the buyer and resets deadline** `KingOfThePoolBlock.processBuy` KingOfThePoolLogic.processBuy updates currentKing even when potContribution is zero, so a buyer can claim the pot without a fee contribution.	**BY-DESIGN**	Dust buys whose nominal fee rounds to 0 cannot crown; non-dust buys crown when the 25% cap is full or seeded in quote currency so the pot remains contestable (KingOfThePoolBlock.sol:154-156).

ID	Conf.	Finding	Status	Current evidence
S3-F45	90	**processWaterfallKing calls processBuy even when potCut rounds to zero** `KingOfThePoolBlock.processWaterfallKing` processWaterfallKing calls processBuy even when potCut is zero, so an attacker can become currentKing and win the jackpot with zero contribution.	**FIXED**	processBuy line 154-156 crowns only if potContribution>0 or the nominal potFee mulDiv is >0, so rounding-to-zero dust buys are not crowned.
S3-F46	90	**Partial tick traversal in _processCrossedLimitOrderTicks inverts reverse swap direction** `LimitOrdersBlock._processCrossedLimitOrderTicks` LimitOrdersLogic.processCrossedTicks stores a partial tick after 512 steps, so a reverse swap reads the wrong direction and fills opposite limit orders.	**FIXED**	processCrossedTicks always stores `lastTicks[poolId] = postSwapTick` (L373), not a partial tick, so direction cannot invert.
S3-F47	90	**Capped 512-tick loop in processCrossedTicks desynchronizes lastTicks** `LimitOrdersBlock.processCrossedTicks` When a swap crosses more than 512 ticks, processCrossedTicks stores an incomplete lastTicks value, and a reversal swap skips valid limit orders.	**FIXED**	lastTicks set to postSwapTick (L373), so no desync. Buckets beyond 512 steps in that swap are still skipped (see S3-L05).
S3-F48	90	**Lagging lastTicks after 512 steps inverts movedUp on the next swap** `LimitOrdersLogic.processCrossedTicks` A swap across more than 512 ticks leaves lastTicks behind postSwapTick, so the next swap reverses movedUp and fills opposite limit orders.	**FIXED**	No lagging lastTicks: L373 writes postSwapTick unconditionally after the capped loop.
S3-F49	90	**Oracle checkpoint integrates pre-swap tick instead of post-swap tick** `OracleBlock._updateOracleCheckpoint` updateCheckpoint integrates obs.lastTick instead of the current pool tick over elapsed time, so the oracle reports the price from before the last swap.	**FIXED**	updateCheckpoint integrates the passed currentTick, not obs.lastTick (OracleBlock.sol:82-86); call site passes the live pre-swap tick.
S3-F50	90	**One-wei-per-second dust orders fill all 256 active TWAMM slots for a year** `TWAMMBlock.submitTWAMMOrder` submitOrder accepts one wei per second up to 256 active orders, so an attacker can fill all order slots with dust and block new orders.	**FIXED**	submitOrder enforces MIN_TWAMM_ORDER_AMOUNT = 1e15 and totalAmount >= durationSeconds (TWAMMBlock.sol:106,202-203).
S3-F51	90	**Uint160 cast before MAX_SQRT_PRICE check wraps graduation price** `VirtualXYKLaunch._graduateToV4` _graduateToV4 casts the shifted square root to uint160 before the MAX_SQRT_PRICE check, so a high token-to-quote ratio wraps the price and makes graduation revert.	**FIXED**	Tiered branches bound the sqrt before the cast (X160 for tokens<=uint96; X128 overflows in mulDiv before a uint160 wrap) (VirtualXYKLaunch.sol:201-216).
S3-F52	90	**High token-to-quote ratio overflows liquidity math in VirtualXYKLaunch.unlockCallback** `VirtualXYKLaunch.unlockCallback` A launch with a high token-to-quote ratio overflows the uint160 cast in _graduateToV4 and fails to settle in unlockCallback.	**FIXED**	Tiered sqrt computation prevents the uint160 wrap (VirtualXYKLaunch.sol:201-216). sqrt(quote*tokens) overflows only beyond realistic supply.
S3-F53	90	**Initial sell in unlockCallback leaves transient balance at zero** `WalletCapBlock._enforceWalletCapAfterSwap` WalletCapLogic.enforceAfterSwap subtracts sold tokens from zero instead of actualBal in TransientContext, so a trader who sells then buys in one unlockCallback reverts.	**FIXED**	WalletCapBlock.sol:96-99: when the cached value is zero, a sell seeds prevExpected from balanceOf(trader) before subtracting sold.

ID	Conf.	Finding	Status	Current evidence
S3-F54	90	**Sell before buy in one unlock callback triggers false WalletCapExceeded revert** `WalletCapBlock.enforceAfterSwap` WalletCapLogic.enforceAfterSwap subtracts sold tokens from zero instead of the token balance on an initial sell, so a subsequent buy in the same transaction reverts.	**FIXED**	Same fix (WalletCapBlock.sol:97-98). The cached value is balanceOf minus sold, not zero, and enforceAfterBuy takes max(actual, cached), so no false revert.
S3-F55	85	**Floor reserve donation via PoolManager.donate lets flash liquidity take reserves** `AutoBuybackBlock.processWaterfallBuyback` AutoBuybackBlock donates reserves via PoolManager.donate, so an attacker who adds flash liquidity at triggerFloorTick takes the reserve in one transaction.	**FIXED**	Donations are queued and flushed in beforeSwap of a later swap, skipped if liquidity was added this block (AutoBuybackBlock.sol:146, Orchestrator:277-279). Same-tx flash liquidity is blocked.
S3-F56	85	**Intra-block swaps overwrite lastObservedTick and dilute dynamic fee volatility** `DynamicFeeBlock._computeDynamicFee` A trader who splits a large trade into smaller swaps in one block advances lastObservedTick each time and lowers the dynamic fee.	**FIXED**	compute only updates lastObservedTick on a new block. Same-block swaps only raise rollingVolatilityTicks (DynamicFeeBlock.sol:93-99).
S3-F57	85	**Three-tick swaps advance lastObservedTick while volatility rounds to zero** `DynamicFeeBlock.compute` A trader who splits a swap into three-tick steps updates state.lastObservedTick while state.rollingVolatilityTicks rounds down to zero, so the trader pays baseFee.	**FIXED**	The EMA adds +3 when absDelta>0 (`((r*3 + absDelta + 3)/4)`), so three-tick moves no longer round volatility to zero (DynamicFeeBlock.sol:94).
S3-F58	85	**Standard pools lack a caller check in _beforeInitialize** `HookRigOrchestrator._beforeInitialize` The _beforeInitialize hook checks the caller only for launchpad pools, so an attacker can initialize any standard pool at a wrong price.	**FIXED**	AccessControlLogic.verifyInitializer rejects any caller other than poolAdmin or orchestrator owner on registered standard pools (AccessControlBlock.sol:49-66).
S3-F59	85	**Swap rewards use raw unspecified token units across swap modes** `HookRigOrchestrator._runAfterSwapPipeline` _runAfterSwapPipeline passes raw unspecifiedCurrency volume to _accrueSwapRewards, so a trader chooses the swap mode with the cheaper token to inflate reward points.	**FIXED**	HookRigOrchestrator.sol:409-411: reward volume is always the project-token leg (unspecified minus cut, or settledSpecifiedAmount), whatever the swap mode.
S3-F60	85	**Zero maxWalletBps default in _validateWalletCapAlignment mismatches WalletCapLogic** `LaunchpadFactory._validateWalletCapAlignment` LaunchpadFactory._validateWalletCapAlignment treats a zero maxWalletBps as 200 bps while WalletCapLogic.init stores zero, so the pool disables the wallet cap after graduation.	**FIXED**	_validateWalletCapAlignment writes maxWalletBps=200 into stackConfig when it is zero, before registerPoolStack (LaunchpadFactory.sol:224-226).
S3-F61	85	**One-tick swap locks opposite-direction swaps for all users in that block** `MEVProtectionBlock._enforceMEVAfterSwap` enforceAfterSwap blocks every opposite-direction swap in the same block globally, so an attacker can make a one-tick swap first to block opposite trades.	**BY-DESIGN**	Swaps below dirThreshold do not lock block direction; threshold-sized opposite-direction lock is the intended anti-sandwich mechanism (MEVProtectionBlock.sol, docs/04).
S3-F62	85	**Pool-wide lastSwapZeroForOne in enforceAfterSwap blocks opposite swaps in the same block** `MEVProtectionBlock.enforceAfterSwap` An attacker who front-runs with a one-tick swap sets lastSwapZeroForOne, which makes every opposite swap in that block revert in MEVProtectionLogic.enforceAfterSwap.	**BY-DESIGN**	Pool-wide intra-block direction lock for swaps >= dirThreshold is the intended anti-sandwich design and is documented in MEVProtectionBlock NatSpec and docs/04.

ID	Conf.	Finding	Status	Current evidence
S3-F63	85	**Unallocated band tokens and remainders stay locked in NativeTickLaunch** `NativeTickLaunch.seedLiquidity` LaunchpadFactory sends the full available token supply to NativeTickLaunch, but seedLiquidity leaves unallocated band tokens and division remainders locked in the contract forever.	**FIXED**	LaunchpadFactory validates all NativeTick bands and NativeTickLaunch burns any residual project tokens to 0xdEaD (LaunchpadFactory.sol:308-355, NativeTickLaunch.sol:102-108).
S3-F64	85	**Base-price redemption cap in SteppedTrancheLaunch.sell locks surcharges and step premiums** `SteppedTrancheLaunch.sell` A buyer who exits a stalled SteppedTrancheLaunch via sell loses all anti-snipe surcharges and tranche price increments, which stay locked in the contract forever.	**FIXED**	SteppedTrancheLaunch tracks buyerSurcharges[msg.sender] and refunds pro-rata surcharges via claimStalledSurchargeRefund in stalled launches (SteppedTrancheLaunch.sol:245-272).
S3-F65	85	**Leftover reserve donation at graduation locks tokens inside PoolManager** `SteppedTrancheLaunch.unlockCallback` SteppedTrancheLaunch and VirtualXYKLaunch pass leftover reserve tokens to poolManager.donate at graduation, which locks those tokens as unclaimable fees inside PoolManager.	**BY-DESIGN**	Leftover donation in unlockCallback (Stepped:295-307) is an accepted design limit: locked graduation LP fees stay in PoolManager (docs/04:91, docs/07 TODO-21).
S3-F66	85	**Integer division remainder in TWAMMLogic.submitOrder is locked forever** `TWAMMLogic.submitOrder` TWAMMLogic.submitOrder rounds the per-second rate down and cancelOrder rejects expired orders, so the hook locks the remainder totalAmount modulo durationSeconds forever.	**FIXED**	claimTWAMMOrder and cancelTWAMMOrder refund totalAmount - swappedInputAmount - refundedInputAmount after expiry, recovering all integer division remainder dust (TWAMMBlock.sol:301,341).
S3-F67	85	**Leftover tokens donated in VirtualXYKLaunch.unlockCallback are locked as unclaimable fees** `VirtualXYKLaunch.unlockCallback` VirtualXYKLaunch and SteppedTrancheLaunch pass leftover tokens to poolManager.donate on a locked LP position, so unsold tokens become unclaimable fees.	**BY-DESIGN**	VirtualXYKLaunch.unlockCallback:252-264 donates leftovers to the locked LP, documented as accepted (docs/04:91; docs/07 TODO-21).
S3-F68	80	**Window rollover sets anchorTick from unverified instantaneous preSwapTick** `CircuitBreakerBlock.checkAndAdvance` checkAndAdvance sets anchorTick to the instantaneous preSwapTick when a window expires, so an attacker can lock anchorTick at an extreme tick.	**BY-DESIGN**	CircuitBreaker rollover anchor is clamped to prevAnchor +/- maxWindowTickDeviation, bounding drift to at most one band per window (CircuitBreakerBlock.sol, docs/04).
S3-F69	80	**VirtualXYKLaunch.buy credits full quoteUsed without a fee-on-transfer balance check** `VirtualXYKLaunch.buy` buy credits the full quoteUsed to realQuoteRaised without a balance check, so a fee-on-transfer quoteCurrency leaves the curve insolvent and blocks _graduateToV4.	**FIXED**	VirtualXYKLaunch.buy measures quote token balance before/after safeTransferFrom and reverts UnsupportedFeeOnTransferQuoteToken on shortfall (VirtualXYKLaunch.sol:120-126).
S3-L01	lead	**Floor defense donates reserves to LPs instead of a buyback swap** `AutoBuybackBlock._processAutoBuyback` Code smells: AutoBuybackLogic.settleLPDonation calls manager.donate(key, amount0, amount1, "") instead of a pool swap, which increases LP fee growth without moving sqrtPriceX96 or tick. — AutoBuybackLogic.settleLPDonation calls PoolManager.donate instead of a swap below triggerFloorTick. We did not verify whether an external contract swaps those fees.	**BY-DESIGN**	flushPendingDonations calls manager.donate (AutoBuybackBlock.sol:154). docs/04 line 123 and section 3.7 document floor defense as a queued LP donation, not a swap.

ID	Conf.	Finding	Status	Current evidence
S3-L02	lead	**Router unlockCallback settles native ETH from contract balance without msg.value check** `HoodiSwapRouter.unlockCallback` Code smells: HoodiSwapRouter.unlockCallback in script/HoodiIntegration.s.sol calls data.key.currency0.settle without verifying msg.value against -delta.amount0(). — HoodiSwapRouter.unlockCallback settles native ETH from the router balance without a msg.value check. We did not verify if the router holds ETH between transactions.	FIXED	HoodiSwapRouter.unlockCallback verifies settle0 <= cb.msgValue before settling native ETH, so router-held ETH is never spent (script/HoodiIntegration.s.sol:69-74).
S3-L03	lead	**Configured jitExitPenaltyBps is never deducted on liquidity removal** `JITGuardLogic.enforceOnRemove` Code smells: JITGuardLogic.enforceOnRemove is a view function and HookRigOrchestrator sets afterRemoveLiquidityReturnDelta = false, so jitExitPenaltyBps only forces minHold = 1. — JITGuardLogic.enforceOnRemove never deducts jitExitPenaltyBps on liquidity removal. Same-block removals revert instead when minLPHoldBlocks is zero.	FIXED	enforceOnRemove enforces minHold = max(minLPHoldBlocks, jitExitPenaltyBps > 0 ? 1 : 0) and docs/04 & NatSpec document that v4 beforeRemoveLiquidity reverts rather than deducting delta.
S3-L04	lead	**Tick loop includes prevTick when prevTick is aligned to tickSpacing** `LimitOrdersBlock._processCrossedLimitOrderTicks` Code smells: LimitOrdersLogic.processCrossedTicks starts at aligned = (prevTick / step) * step, which equals prevTick when prevTick % step == 0. — The tick loop checks prevTick when prevTick is a multiple of tickSpacing. We did not verify an active override of _swapFilledOrderOnClaim.	BY-DESIGN	Inclusive aligned start tick on departure crossing is idempotent with bucket epoch tracking and documented in LimitOrdersBlock NatSpec.
S3-L05	lead	**Large swap followed by reverse swap skips buckets beyond 512 ticks** `LimitOrdersBlock._processLimitOrderTicks` Code smells: LimitOrdersLogic.processCrossedTicks caps the loop at MAX_TICK_FILLS_PER_SWAP = 512 and stores lastReached in lastTicks[poolId]. — A large swap followed by a reverse swap skips limit order buckets beyond 512 ticks. We did not verify the gas cost for 512 ticks.	FIXED	Reverse-swap lastTicks desync is fixed and processCrossedTicks anchors lastTicks at the last inspected tick when MAX_TICK_FILLS_PER_SWAP=512 is hit (LimitOrdersBlock.sol:378-406).
S3-L06	lead	**Virtual order execution advances timestamp without a swap of maker tokens** `TWAMMBlock._executeTWAMM` Code smells: TWAMMBlock._executeTWAMM advances lastVirtualOrderTimestamp and emits TWAMMVirtualOrdersExecuted without calling poolManager.swap or crediting maker output. — Every swap advances lastVirtualOrderTimestamp in TWAMMLogic.executePending without a swap of the locked maker tokens. No claim function exists in TWAMMBlock.	FIXED	_beforeSwap calls _syncOracleAndTWAMM -> _executePendingTWAMMOrders(key, poolId) which swaps accrued virtual volume against PoolManager and credits maker claimableOutput (HookRigOrchestrator.sol:299-316).

8. Scan 4 — 2026-10-15 10:03 (594edcd)

3-pass scan of commit `594edcd` (2026-10-15 3-pass loop). Remediation: 03fd355 (all 35 findings, 22 leads & TODO-14..29 remediated). 35 findings and 22 leads; current status: 52 fixed, 5 by-design.

ID	Conf.	Finding	Status	Current evidence
S4-F01	100	**Order maker re-enters PoolManager.swap while every hook check is off** `HookRigOrchestrator._beforeSwap` · seen in 3/3 runs · NEW The internal-swap flag covers the whole pool and `claimLimitOrder` has no reentrancy guard, so a maker who gets the ETH payout swaps with no fee or check.	**FIXED**	Internal-execution flag removed; claim swaps run the full hook pipeline (HookRigOrchestrator._beforeSwap/_afterSwap); claimLimitOrder is nonReentrant. Test test_Fix_ClaimSwapPaysCutsAndCreditsMaker.
S4-F02	95	**Claim swap pays no LP fee and skips every hook cut** `LimitOrdersBlock.claimLimitOrder` · seen in 3/3 runs · NEW The claim swap returns fee 0 with no override flag, so on a dynamic-fee pool the maker pays a zero LP fee and no hook cut.	**FIXED**	Claim swap pays the resolved LP fee and every return-delta cut; executor reports the maker via IMsgSender so fees/points/crowns go to the maker.
S4-F03	95	**The daily sell cap reads a balance the seller controls** `VestingSellCapBlock.enforceSellCap` · seen in 3/3 runs · KNOWN (2 scans) The epoch balance comes from `balanceOf(seller)` at the first sell, so router sells revert and a seller who borrows or moves in tokens first sells far past the cap.	**FIXED**	_preSellBalance nets PoolManager deltas; snapshot lowered when tokens leave; zero-balance untrusted routers skipped instead of bricked. Tests test_Fix_VestingSellCap_*.
S4-F04	90	**First buyer after a stepped graduation keeps the crown and takes the seeded pot** `KingOfThePoolBlock.processBuy` · seen in 2/3 runs · KNOWN (2 scans) The launch seed puts the pot in the quote currency, so later project-token buys add no pot and cannot take the crown.	**FIXED**	Non-dust buys take the crown whenever a pot exists, even with zero pot cut; dust (nominal fee 0) never crowns. Test test_Fix_KingSeedInOtherCurrencyStillContestable.
S4-F05	90	**Attacker copies a pending launch salt and takes the announced token address** `LaunchpadFactory._deployTokenAndVest` · seen in 1/3 runs · NEW The CREATE2 salt does not include `msg.sender`, so an attacker who copies a pending launch deploys the token first with the attacker's pool admin, fee recipients and curve settings.	**FIXED**	CREATE2 salt = keccak256(msg.sender, salt) in _deployTokenAndVest and predictTokenAddressFor. Test test_Fix_LaunchSaltIsBoundToSender.
S4-F06	85	**Exact-output buys skip the auto burn and the King pot fee** `AfterSwapDeltaAggregator._executeAfterSwapDeltaWaterfall` · seen in 1/3 runs · KNOWN (2 scans) The burn and King steps require the project token as the unspecified currency, so a buyer who uses exact output pays neither cut.	**FIXED**	Exact-output buys revert ExactOutputBuyNotSupported on AutoBurn/King pools (AfterSwapDeltaAggregator).
S4-F07	85	**Attacker registers the predicted launch pool first and blocks the launch** `HookRigOrchestrator.registerPoolStack` · seen in 1/3 runs · KNOWN (2 scans) `initPoolMeta` checks the factory only when `authorizedCurve` is set, so an attacker who registers the predicted launch pool makes the launch revert and holds its pool admin role.	**FIXED**	initPoolMeta rejects pools whose tokens have no code (TokenNotDeployed), so a predicted launch pool cannot be pre-registered. Test test_Fix_RegisterPoolStackRejectsUndeployedToken.
S4-F08	85	**Add from one address and remove from another skips the JIT hold** `JITGuardBlock.recordAddition` · seen in 1/3 runs · KNOWN (2 scans) For salt 0 the guard key includes the resolved user, so an attacker adds through a trusted router as one address and removes as another in the same block.	**FIXED**	JITGuardBlock tracks per-key lpTrackedLiquidity; removals where lpTrackedLiquidity[ownerKey] < removedLiq fall back to posKey hold enforcement (JITGuardBlock.sol:34-155).

ID	Conf.	Finding	Status	Current evidence
S4-F09	85	**Creator launches with a custom hook that blocks buyer sells** `LaunchpadFactory.launchVirtualXYK` · seen in 1/3 runs · KNOWN (2 scans) The factory accepts any `hookContract`, so a creator graduates the curve into a pool whose hook rejects buyer sells, and the creator sells into the locked LP.	**FIXED**	_resolveHook accepts only defaultOrchestrator (UnsupportedHookContract). Test test_Fix_LaunchRejectsUnsafeStacksAndHooks.
S4-F10	85	**Claim swap has no price limit and no minimum output** `LimitOrdersBlock.claimLimitOrder` · seen in 2/3 runs · NEW The executor swaps with the widest price limit and checks only `outAmount > 0`, so an attacker who trades around the claim keeps most of the maker output.	**FIXED**	New claimLimitOrder(key, orderId, minAmountOut) overload reverts ClaimOutputBelowMinimum. Test test_Fix_ClaimSwapMinimumOutput.
S4-F11	80	**Ending a swap outside all ranges skips the ArbRecapture cut** `AfterSwapDeltaAggregator._executeAfterSwapDeltaWaterfall` · seen in 1/3 runs · NEW The hook checks liquidity only at the post-swap tick, so an arbitrageur who ends one tick past the last range pays no ArbRecapture or LPRewards cut.	**FIXED**	ArbRecapture and LPRewards cuts no longer depend on post-swap liquidity; they are always taken and queued.
S4-F12	80	**A trader takes back the ArbRecapture and LPRewards donation** `AfterSwapDeltaAggregator._executeAfterSwapDeltaWaterfall` · seen in 3/3 runs · NEW The cuts are donated to in-range liquidity inside afterSwap, so a trader who adds narrow liquidity at the post-swap tick in the same unlock gets most of them back.	**FIXED**	LP donations queued (pendingLPDonation) and flushed in beforeSwap of a later swap in a block with no liquidity add. Test test_Fix_LPDonationWaitsForBlockWithoutLiquidityAdds.
S4-F13	80	**One reserve threshold for both currencies locks the other reserve forever** `AutoBuybackBlock.accumulateAndCheckTrigger` · seen in 1/3 runs · NEW The trigger compares both currency reserves with one raw `minBuybackReserve`, so the reserve in the smaller-unit currency never triggers and no function releases it.	**FIXED**	minBuybackReserve is quote-denominated; on trigger both currency reserves move into the queued donation.
S4-F14	80	**Narrow LP added one transaction earlier takes the buyback donation** `AutoBuybackBlock.accumulateAndCheckTrigger` · seen in 1/3 runs · NEW The only anti-JIT guard uses transient storage, so an attacker adds narrow liquidity in one transaction and triggers the floor donation in the next to take most of the reserve.	**FIXED**	Transient same-tx guard replaced by a block-scoped _lastLiquidityAddBlock check before any donation is paid.
S4-F15	80	**Preset hooks accept blocks whose liquidity callbacks they never receive** `AccessControlLogic.validateBitmapAndPermissions` · seen in 3/3 runs · KNOWN (2 scans) The validator uses a wrong full-flags constant and checks only two flags, so JITGuard, AccessControl and the AutoBuyback same-transaction guard never run on preset hooks.	**FIXED**	validateBitmapAndPermissions requires the hook address to carry every flag permissionsForBitmap(bitmap) needs. Test test_Fix_PresetHookRejectsBlocksWithoutCallbacks.
S4-F16	80	**Creator enables AccessControl so launch buyers cannot sell** `LaunchpadFactory._deploySortedTokenAndRegisterPool` · seen in 1/3 runs · NEW The factory accepts a stack with AccessControl and an empty root, so after graduation only the creator can sell into the locked LP.	**FIXED**	Factory rejects BIT_ACCESS_CONTROL on launch stacks (UnsafeLaunchStack).

ID	Conf.	Finding	Status	Current evidence
S4-F17	80	**Creator restarts an 80% anti-snipe fee at graduation and takes holder sales** `LaunchpadFactory._deploySortedTokenAndRegisterPool` · seen in 1/3 runs · NEW The anti-snipe fee is outside the 25% budget and restarts at graduation, so a creator with narrow liquidity takes most of each holder's sale.	FIXED	Factory caps AntiSnipe launchFee at MAX_LAUNCH_ANTI_SNIPE_FEE = 25% on launch stacks.
S4-F18	80	**A tick spacing above 32767 makes graduation revert forever** `LaunchpadFactory._deploySortedTokenAndRegisterPool` · seen in 1/3 runs · NEW The factory checks only the lower bound of `tickSpacing`, so every graduating buy reverts in `PoolManager.initialize` and buyer quote stays in the curve.	FIXED	Factory reverts when tickSpacing > TickMath.MAX_TICK_SPACING for every curve type.
S4-F19	80	**A 1-wei first swap blocks opposite trades for the block** `MEVProtectionBlock.enforceAfterSwap` · seen in 1/3 runs · KNOWN (2 scans) The tracker records the direction of any first swap in a block, so an attacker's 1-wei swap makes larger opposite trades revert for that block.	FIXED	Only swaps of at least dirThreshold set the block direction (blockDirectionSet). Test test_Fix_MEVDustSwapDoesNotSetBlockDirection.
S4-F20	80	**A rounding overshoot opens the stepped pool at the next tranche price** `SteppedTrancheLaunch._graduateToV4` · seen in 1/3 runs · NEW The final buy sells a few wei past the tranche boundary, so graduation prices the pool at the next tranche and the pool opens 20% above the price buyers paid.	FIXED	Graduation opens at lastFillPriceQ96 (price of the last curve fill), not the rounded tranche index. Fuzz repro GL-11 passes.
S4-F21	80	**Graduation clamps LP tokens but keeps all quote, so the pool opens above the curve price** `VirtualXYKLaunch._graduateToV4` · seen in 1/3 runs · NEW When `virtualTokenReserve` exceeds the real token supply, graduation seeds fewer tokens with the full quote, so an early buyer sells above the curve price and takes LP quote.	FIXED	When LP tokens are clamped, quoteToSeed is scaled proportionally so the pool opens at the curve price.
S4-F22	80	**The curve LP position cannot collect its fees** `VirtualXYKLaunch.unlockCallback` · seen in 2/3 runs · NEW Traders pay LP fees and LP-reward donations to the curve's position, but no function collects them, so the tokens stay in PoolManager.	BY-DESIGN	Curve-owned graduation LP is locked by design (equivalent of burned LP); documented in docs/04 section 3.7 and TODO-21.
S4-F23	75	**AutoBuyback does not request beforeAddLiquidity** `HookPermissionMasks.permissionsForBitmap` · seen in 1/3 runs · KNOWN (2 scans) The `beforeAddLiquidity` mask leaves out `BIT_AUTO_BUYBACK`, so on preset hooks an LP adds liquidity and takes the floor donation in one transaction.	FIXED	permissionsForBitmap requests beforeAddLiquidity for AutoBuyback/ArbRecapture/LPRewards and wallet-rule blocks, and beforeSwap for donation blocks.
S4-F24	75	**One-sided liquidity lets traders skip the sell cap, buy cap and taxes** `HookRigOrchestrator._beforeAddLiquidity` · seen in 2/3 runs · NEW A trader uses a one-sided range order instead of a swap, so the vesting cap, the wallet cap and the buy and sell cuts never run.	FIXED	enforceTwoSidedLiquidity rejects one-sided adds on WalletCap/VestingSellCap pools (curve exempt). Test test_Fix_WalletCapPoolRejectsOneSidedLiquidity.
S4-F25	75	**Pool admin takes the anti-snipe surcharge when King is off** `HookRigOrchestrator.seedKingPotFromLaunchCurve` · seen in 2/3 runs · KNOWN (2 scans) The seed falls back to the pool admin's claim when the pool has no King block, so the admin takes the buyers' surcharge.	FIXED	seedPotFromLaunchCurve never credits the pool admin; SteppedTrancheLaunch seeds the pot only when King is enabled, otherwise all surcharge goes to LP.

ID	Conf.	Finding	Status	Current evidence
S4-F26	75	**A pool admin raises the tax from 0% to 10% after buyers hold the token** `HookRigOrchestrator.setBuySellTax` · seen in 1/3 runs · NEW `decayOnly` is optional and the factory never sets it, so the admin raises the tax after launch and takes 10% of every swap.	FIXED	BuySellTaxLogic.update forbids any tax increase regardless of decayOnly. Test test_Fix_TaxCannotIncreaseWithoutDecayOnly.
S4-F27	75	**ArbRecapture fills the cut cap so large buyers never take the crown** `KingOfThePoolBlock.processWaterfallKing` · seen in 1/3 runs · NEW The tax budget does not count the ArbRecapture cut, so a large buy fills the 25% cap before the King step and the current king keeps the pot.	FIXED	Crown rule no longer needs a pot cut (non-dust buys crown when a pot exists). ArbRecapture is still not counted in validateTaxBudget (TODO-20).
S4-F28	75	**A zero pot fee locks the launch seed in the hook** `KingOfThePoolBlock.seedPotFromLaunchCurve` · seen in 2/3 runs · NEW The curve seeds the King pot when the pot fee is zero, so no buyer ever becomes king and the seeded quote stays locked in the hook.	FIXED	initKing requires potFeeOnBuyBps > 0; seeds into a zero-fee pot are impossible. Test test_Fix_KingRequiresPotFee.
S4-F29	75	**launchNativeTick skips the wallet-cap default** `LaunchpadFactory.launchNativeTick` · seen in 1/3 runs · NEW `launchNativeTick` does not call `_validateWalletCapAlignment`, so a zero `maxWalletBps` turns off the wallet cap and the first buyer can take the whole band supply.	FIXED	launchNativeTick calls _validateWalletCapAlignment(stackConfig, 0) so a zero cap defaults to 200 bps.
S4-F30	75	**NativeTick band positions cannot collect their fees** `NativeTickLaunch.unlockCallback` · seen in 1/3 runs · NEW The curve owns every band position and has no collect path, so all LP fees, including the anti-snipe fee, stay in PoolManager.	BY-DESIGN	Curve-owned NativeTick band positions are locked by design; documented in docs/04 section 3.7 and TODO-21.
S4-F31	75	**The oracle weights each interval by the previous interval's tick** `OracleBlock.updateCheckpoint` · seen in 1/3 runs · KNOWN (2 scans) The checkpoint multiplies the elapsed time by the old `lastTick`, so an attacker who moves the price for one block skews the TWAP for the whole next interval.	FIXED	Cumulative adds clamped current pre-swap tick x elapsed; live extrapolation uses the clamped slot0 tick. Test test_Fix_OracleWeightsIntervalByItsOwnTick.
S4-F32	75	**Dust TWAMM orders fill the per-pool order cap for a year** `TWAMMBlock.submitTWAMMOrder` · seen in 1/3 runs · NEW The 256-order cap is shared and has no real minimum size, so an attacker opens 256 dust orders and blocks every other TWAMM order on the pool.	FIXED	submitOrder enforces MIN_TWAMM_ORDER_AMOUNT = 1e15 alongside MAX_ACTIVE_TWAMM_ORDERS = 256 (TWAMMBlock.sol:106-107). Test test_Block16_TWAMM_OrdersDisabled.
S4-F33	70	**TWAMM deposits never swap and have no output claim** `TWAMMBlock.submitTWAMMOrder` · seen in 2/3 runs · KNOWN (2 scans) `executePending` only counts virtual volume, so a TWAMM user never receives output tokens.	FIXED	TWAMMBlock.sol executes virtual order swaps in _beforeSwap and via TWAMMSwapExecutor and pays makers via claimTWAMMOrder / cancelTWAMMOrder. Test test_Block16_TWAMM_OrdersDisabled.
S4-F34	65	**Moving tokens to new wallets resets the daily sell cap** `VestingSellCapBlock._enforceVestingSellCap` · seen in 1/3 runs · NEW The epoch is keyed by the selling address, so a holder who moves the rest to a new wallet after each sell sells most of the holding in one day.	BY-DESIGN	Per-wallet caps cannot bind a holder who splits tokens across wallets; documented in docs/04 section 3.7 and TODO-21.

ID	Conf.	Finding	Status	Current evidence
S4-F35	65	**Batched buys on two pools of one token get twice the wallet cap** `WalletCapBlock.enforceAfterBuy` · seen in 1/3 runs · NEW The wallet-cap cache is keyed by pool and not by token, so a buyer who batches buys on two pools of one token gets twice the cap.	**FIXED**	Wallet-cap transient cache keyed by project token instead of PoolId, so batched buys across pools of one token add up.
S4-L01	lead	**An allowlisted untrusted router admits every caller** `AccessControlBlock.verifyAccess` Code smells: the check uses the router address, not the user — We did not confirm a deployment that allowlists such a router.	**FIXED**	AccessControlLogic.verifyAccess rejects trusted routers that self-attribute (user == sender && _isTrustedRouter(user)) and requires end-user identity (AccessControlBlock.sol:178-190).
S4-L02	lead	**A short burn take spends other users' escrow** `AutoBurnBlock.execute` Code smells: the burn uses the hook's shared ERC20 balance — We did not find a token that causes the shortfall.	**FIXED**	AutoBurnLogic.takeAndBurn measures actual ERC-20 balance received around manager.take and burns only received tokens (AutoBurnBlock.sol:63-79).
S4-L03	lead	**The deploy script falls back to a public Anvil key** `HoodiIntegrationScript.run` Code smells: `vm.envOr("PRIVATE_KEY", <Anvil key #0>)` with `--broadcast` on Hoodi — Anyone who knows that key owns the orchestrator. We did not confirm a broadcast with that key.	**FIXED**	HoodiIntegration.s.sol requires vm.envUint('PRIVATE_KEY') on all non-local chains (block.chainid != 31337) with no fallback key (script/HoodiIntegration.s.sol:122-124).
S4-L04	lead	**The script router pays native input from its own ETH** `HoodiSwapRouter.swap` Code smells: `swap` does not check that `msg.value` covers the input — Impact needs ETH that already sits in the router.	**FIXED**	HoodiSwapRouter passes msg.value in CallbackData and reverts InsufficientNativeValue if settle0 > cb.msgValue (script/HoodiIntegration.s.sol:49-85).
S4-L05	lead	**Per-pool claims draw on one shared ERC-6909 balance** `HookRigBase.withdrawClaim` Code smells: no per-pool backing total — Any unbacked credit in one pool burns other pools' backing. We found only the owner path.	**FIXED**	setLaunchpadFactory is set-once (LaunchpadFactoryAlreadySet) and KingOfThePoolLogic.validateClaim caps payouts at available ERC-6909 balance (HookRigBase.sol:86-91, KingOfThePoolBlock.sol:251-271).
S4-L06	lead	**Exact-output sells earn fewer swap points** `HookRigOrchestrator._runAfterSwapPipeline` Code smells: volume is `gross - cut` where the trader pays `gross + cut` — The same sale earns different points by mode. We found no payout that uses the points.	**FIXED**	HookRigOrchestrator._runAfterSwapPipeline adds totalCutAmount to settledSpecifiedAmount on exact-output sells so SwapRewards credits full project-token volume (HookRigOrchestrator.sol:420-430).
S4-L07	lead	**A launchpad factory address set by the owner can credit unbacked claims** `HookRigOrchestrator.seedKingPotFromLaunchCurve` Code smells: no ERC-6909 backing check; `launchpadFactory` can be changed at any time — The owner can withdraw other pools' claims. The deploy script's public fallback key may make this reachable by anyone; we did not confirm a live deployment with that key.	**FIXED**	setLaunchpadFactory is locked after initial setup (LaunchpadFactoryAlreadySet) and rejects address(0). Test test_Fix_SetLaunchpadFactory_OneTimeLockAndStandardPoolInitAuth.
S4-L08	lead	**A 20-wei buy takes the crown when the minimum is zero** `KingOfThePoolBlock.initKing` Code smells: `minBuyAmountForReset = 0` is accepted — Any buyer can take the crown with dust just before the deadline. We did not show this is outside the game design.	**BY-DESIGN**	minBuyAmountForReset = 0 is an optional creator configuration while nominal potFee > 0 still blocks 1-wei rounding-to-zero dust buys (KingOfThePoolBlock.sol:154-156).
S4-L09	lead	**The seed credit does not check that the hook received ERC-6909** `KingOfThePoolBlock.seedPotFromLaunchCurve` Code smells: the function trusts `launchpadFactory` for every pool — Only an owner-set factory reaches this. The shipped curves always mint before they seed.	**FIXED**	seedPotFromLaunchCurve requires Round 1 with currentKing == address(0) and validateClaim caps reward at PoolManager ERC-6909 balance (KingOfThePoolBlock.sol:251-290).

ID	Conf.	Finding	Status	Current evidence
S4-L10	lead	**A zero creator makes the factory the pool admin** `LaunchpadFactory._deploySortedTokenAndRegisterPool` Code smells: admin falls back to `msg.sender`, the factory — The surcharge seed goes to a claim nobody can withdraw. We did not prove a third party profits.	**FIXED**	LaunchpadFactory._resolveCreator defaults address(0) creator to msg.sender instead of address(this) (LaunchpadFactory.sol:202-204).
S4-L11	lead	**launchVirtualXYK accepts a tick spacing the PoolManager rejects** `LaunchpadFactory.launchVirtualXYK` Code smells: only values at or below 0 are replaced — Buyers can only exit through `sell`. We did not find a party who gains.	**FIXED**	Factory rejects tickSpacing > MAX_TICK_SPACING for every launch type.
S4-L12	lead	**A maker can neither claim nor cancel with no liquidity in range** `LimitOrdersBlock.claimLimitOrder` Code smells: a zero-output claim reverts and cancel rejects filled buckets — We did not prove a lock longer than the time until new liquidity arrives.	**FIXED**	Makers can cancel filled orders, so a zero-output claim can no longer lock funds.
S4-L13	lead	**A 1-wei order marks a filled bucket unfilled** `LimitOrdersBlock.placeLimitOrder` Code smells: `placeOrder` resets `filled` when the price is back on the far side — Earlier makers cannot claim until the price crosses again. They can still cancel.	**FIXED**	LimitOrdersBlock tracks uint64 epoch on buckets and orders plus totalInputAmount and epochInputAmount; re-opening a bucket increments epoch so prior epoch orders remain claimable (LimitOrdersBlock.sol:95-225). Test test_Fix_LimitOrders_BucketEpochPrevents1WeiClaimDenial.
S4-L14	lead	**A round-trip swap marks a bucket filled** `LimitOrdersBlock.processCrossedTicks` Code smells: any crossing sets `filled`, even if the price comes back in the same transaction — Makers then lose cancel. We did not show a profit for the attacker.	**FIXED**	A round-trip fill no longer removes the maker's cancel path (cancel works after fill).
S4-L15	lead	**zeroForOne orders fill when the price falls** `LimitOrdersBlock.processCrossedTicks` Code smells: downward crossings fill token0-input buckets — The order acts as a stop order. We did not confirm the intended design.	**BY-DESIGN**	Stop/trigger crossing semantics (zeroForOne fills on downward crossing, !zeroForOne on upward crossing) are documented in LimitOrdersBlock NatSpec and docs/03.
S4-L16	lead	**A large swap skips buckets after 512 ticks** `LimitOrdersBlock.processCrossedTicks` Code smells: the loop stops at 512 steps and then sets `lastTicks` to the post-swap tick — Orders past the cap stay unfilled. Makers can cancel, so we found no loss.	**FIXED**	processCrossedTicks anchors lastTicks[poolId] to the last inspected tick when MAX_TICK_FILLS_PER_SWAP = 512 is reached so subsequent swaps fill remaining buckets (LimitOrdersBlock.sol:378-406).
S4-L17	lead	**Surcharge quote can make the stepped pool open above the curve price** `SteppedTrancheLaunch._graduateToV4` Code smells: token amount is clamped, quote amount is not — We did not build numbers under the factory caps.	**FIXED**	SteppedTrancheLaunch._graduateToV4 scales quoteToSeed proportionally when continuousLpTokens > availableTokens and donates excess quote to LP (SteppedTrancheLaunch.sol:288-371).
S4-L18	lead	**Fee-on-transfer quote tokens make stepped graduation revert** `SteppedTrancheLaunch.buy` Code smells: `realQuoteRaised` adds the requested amount, not the received amount — We did not test a specific token.	**FIXED**	SteppedTrancheLaunch.buy measures quote token balance before/after safeTransferFrom and reverts UnsupportedFeeOnTransferQuoteToken on shortfall (SteppedTrancheLaunch.sol:156-162).
S4-L19	lead	**A snapshot without afterSwap stays for later swaps** `TransientContext.storePreSwapSnapshot` Code smells: the snapshot clears only in afterSwap — Only a hook without afterSwap is affected. We found no such deployed hook.	**FIXED**	TransientContext.loadAndClearPreSwapSnapshot validates snap.blockNumber == uint32(block.number) and HookPermissionMasks always pairs afterSwap with beforeSwap (TransientContext.sol:52-55).

ID	Conf.	Finding	Status	Current evidence
S4-L20	lead	**The init accepts sell caps outside the documented range** `VestingSellCapBlock._initVestingSellCap` Code smells: accepts 50 to 10_000 bps, docs say at most 5000 — We did not confirm the intended policy.	**FIXED**	VestingSellCapLogic.init enforces MAX_DAILY_SELL_BPS = 5000 (50.00%) matching the specification (VestingSellCapBlock.sol:20,40).
S4-L21	lead	**Fee-on-transfer quote tokens make graduation revert** `VirtualXYKLaunch.buy` Code smells: `realQuoteRaised` adds the requested amount, not the received amount — Graduation asks for more quote than the curve holds. We did not confirm a realistic quote token.	**FIXED**	VirtualXYKLaunch.buy verifies exact ERC-20 quote balance delta received and reverts UnsupportedFeeOnTransferQuoteToken (VirtualXYKLaunch.sol:120-126).
S4-L22	lead	**Untrusted routers let a buyer pass the wallet cap each transaction** `WalletCapBlock.enforceAfterBuy` Code smells: the check reads the router balance, which is zero at afterSwap — A buyer who uses an untrusted router can buy up to the cap in every transaction. We did not rate this above the transfer bypass.	**FIXED**	WalletCapLogic.enforceAfterSwap rejects untrusted routers (trader == sender) with UntrustedRouterNotAllowed ForWalletCap. Test test_Fix_UntrustedR outerGuards_WalletCap_King_Referral.

9. Scan 5 — 2026-10-16 07:12 (03fd355)

3-pass scan of commit `03fd355` (2026-10-16 3-pass loop). Remediation: 92ea50b (all 34 findings & 18 leads across 3 passes remediated). 34 findings and 18 leads; current status: 20 fixed, 32 by-design.

ID	Conf.	Finding	Status	Current evidence
S5-F01	96	**Preset 1..11 pools skip `beforeInitialize`, allowing unconfigured `initialize` to anchor `AntiSnipe` and `WalletCap` clocks at `block.number`** `HookRigOrchestrator._afterInitialize · unanchored-standard-pool-clocks-on-init` · seen in 1/3 runs · NEW `_afterInitialize` previously checked `verifyInitializer` (a no-op when `poolAdmin == address(0)`) without checking `meta.configured`, allowing unconfigured `initialize` on Presets 1..11.	FIXED	HookRigOrchestrator._verifyPoolInit enforces `if (!meta.configured) revert PoolNotConfigured(poolId)` in both `_beforeInitialize` and `_afterInitialize` (HookRigOrchestrator.sol:191-202).
S5-F02	95	**`TWAMMLogic.submitOrder` accepts native currency, fee-on-transfer tokens, and unbounded active orders without escrow or loop bounds** `TWAMMBlock.submitOrder · unescrowed-native-and-fot-input` · seen in 1/3 runs · NEW Pass 1 identified missing `payable` / `msg.value` checks, fee-on-transfer balance-delta verification, and active order array bounds in `submitOrder`.	FIXED	TWAMMLogic.submitOrder enforces `msg.value == totalAmount` for native currency, pre/post balance delta for ERC-20s, `MAX_ACTIVE_TWAMM_ORDERS = 256`, and `MIN_TWAMM_ORDER_AMOUNT = 1e15` (TWAMMBlock.sol:106-107,200-224).
S5-F03	95	**`TWAMMLogic.executePendingWithSwap` swaps opposing directions sequentially through AMM fees instead of netting opposing virtual flows first** `TWAMMBlock.executePendingWithSwap · unnetted-bidirectional-virtual-swaps` · seen in 1/3 runs · NEW Pass 1 identified that simultaneous 0->1 and 1->0 TWAMM orders paid double AMM spread and hook cuts instead of netting opposing virtual flows at spot price.	FIXED	TWAMMLogic._netOpposingFlows nets opposing virtual order flows symmetrically at `slot0` spot `sqrtPriceX96` before swapping only the net residual (TWAMMBlock.sol:458-515).
S5-F04	95	**`SteppedTrancheLaunch.sell` allows sniper surcharge refunds to be extracted during active bonding or drained across multiple buyers** `SteppedTrancheLaunch.sell · premature-and-unbacked-surcharge-refund` · seen in 1/3 runs · NEW Pass 1 identified that inline surcharge refunds inside `sell()` could be triggered mid-launch before knowing whether the curve stalled.	FIXED	Removed inline surcharge refunds from `sell()` and added dedicated `claimStalledSurchargeRefund` gated on `block.number > launchCreationBlock + antiSnipeBlocks + MAX_ANTI_SNIPE_BLOCKS` and `curveTokensSold == 0`, refunding the caller's full `buyerSurcharges[msg.sender]` (SteppedTrancheLaunch.sol:245-268).
S5-F05	95	**In-unlock `TWAMMLogic._swapVirtualInput` triggers `HookRigOrchestrator._beforeSwap` and `_afterSwap` as `address(this)`, applying user protections and taxes to virtual orders** `TWAMMBlock._swapVirtualInput · twamm-self-swap-hook-collision` · seen in 1/3 runs · NEW Pass 1 identified that TWAMM virtual swaps needed exemption from user-level `AccessControl`, `MEVProtection`, `WalletCap`, `VestingSellCap`, and `SwapRewards`.	FIXED	HookRigOrchestrator._isTWAMMInternal (sender) exempts `address(this)` and `_twammExecutor` from user-level access, MEV, circuit-breaker, wallet-cap, vesting-cap, and swap-reward checks (HookRigOrchestrator.sol:320-439).

ID	Conf.	Finding	Status	Current evidence
S5-F06	94	`JITGuardLogic.recordAddition` **skips** `lpTrackedLiquidity` **updates on dust deposits and keys by** `sender` **vs** `lpOwner` **asymmetrically** `JITGuardBlock.recordAddition · position-owner-mismatch-and-cooldown-reset` · seen in 2/3 runs · NEW Pass 2 found that `_trackAddKey` skipped `lpTrackedLiquidity[poolId][key] = existing + addedLiq` when `isDust` was true, leaving tracked liquidity below actual position liquidity.	**FIXED**	JITGuardLogic._trackAddKey always increments `lpTrackedLiquidity[poolId][key] = existing + addedLiq` while only skipping the `lpAddedBlock` timestamp reset on dust adds (JITGuardBlock.sol:34-48).
S5-F07	94	`HookPermissionMasks.permissionsForBitmap` **omits** `BIT_TWAMM` **from** `beforeRemoveLiquidity`**, allowing LPs to pull liquidity before virtual orders settle** `HookPermissionMasks.permissionsForBitmap · missing-hook-permission-flags` · seen in 2/3 runs · NEW Pass 2 & 3 found that a TWAMM-only pool without Oracle/MEV/JITGuard did not set `beforeRemoveLiquidity = true` in `permissionsForBitmap`.	**FIXED**	HookPermissionMasks.permissionsForBitmap now includes `BIT_TWAMM` in `perms.beforeRemoveLiquidity` (HookPermissionMasks.sol:193-194).
S5-F08	93	`AfterSwapDeltaAggregator._executeAfterSwapDeltaWaterfall` **skips** `KingOfThePool` **when** `attributedTrader == address(0)`**, letting untrusted routers bypass the pot fee** `AfterSwapDeltaAggregator._executeAfterSwapDeltaWaterfall · untrusted-router-king-fee-bypass` · seen in 2/3 runs · NEW Pass 2 & 3 found that gating `BIT_KING_OF_THE_POOL` on `attributedTrader != address(0)` let buyers route through untrusted routers to avoid paying `potFeeOnBuyBps` during active King rounds.	**FIXED**	AfterSwapDeltaAggregator now invokes `processWaterfallKing` for all buys, and `KingOfThePoolLogic` collects `potCut` from unattributed routers whenever `st.currentKing != address(0)` without crowning the router (AfterSwapDeltaAggregator.sol:158, KingOfThePoolBlock.sol:225-232).
S5-F09	93	`LimitOrdersBlock.placeOrder` **and** `claimOrder` **zero** `bucket.totalInputAmount` **across epochs, breaking escrow accounting and** `LimitOrderTickCrossed` **volume** `LimitOrdersBlock.placeOrder · cross-epoch-bucket-input-corruption` · seen in 1/3 runs · NEW Pass 1 found that zeroing `bucket.totalInputAmount` when re-opening a bucket violated `GL-20` when prior-epoch filled orders were still unclaimed.	**FIXED**	LimitOrdersBlock now tracks both `totalInputAmount` (all open escrowed input across epochs for `GL-20`) and `epochInputAmount` (current-epoch active order volume) (LimitOrdersBlock.sol:31-35,193-336).
S5-F10	92	`LimitOrdersLogic.processCrossedTicks` **overwrites** `lastTicks[poolId] = postSwapTick` **after hitting** `MAX_TICK_FILLS_PER_SWAP`**, permanently skipping uninspected buckets** `LimitOrdersBlock.processCrossedTicks · tick-fill-cap-skips-crossed-buckets` · seen in 3/3 runs · NEW All 3 passes noted that when a swap spans more than `MAX_TICK_FILLS_PER_SWAP = 512` tick spacings, setting `lastTicks[poolId] = postSwapTick` skipped the remaining ticks.	**FIXED**	LimitOrdersLogic.processCrossedTicks now sets `lastTicks[poolId]` to the last inspected tick (`t - step` or `t + step`) when the 512-step bound is reached so subsequent swaps process the remaining ticks (LimitOrdersBlock.sol:378-406).
S5-F11	92	`TWAMMLogic._netOpposingFlows` **nets the entire** `totalUnswapped` **of the smaller side even when extreme** `sqrtPriceX96` **truncates the opposite side's** `equiv` **to 1 wei** `TWAMMBlock._netOpposingFlows · asymmetric-opposing-flow-netting` · seen in 2/3 runs · NEW Pass 2 & 3 found that `_netOpposingFlows` debited the full `totalUnswapped0` even when integer truncation reduced `equiv1` to a tiny fraction.	**FIXED**	TWAMMLogic._netOpposingFlows now recomputes `netted0` from `netted1` (and `netted1` from `netted0`) symmetrically and returns `(0, 0)` if either side rounds to zero (TWAMMBlock.sol:487-516).

ID	Conf.	Finding	Status	Current evidence
S5-F12	92	**In-unlock TWAMM virtual swaps in `_beforeSwap` do not call `_processCrossedLimitOrderTicks`, allowing TWAMM price movement to cross limit-order ticks without filling them** `HookRigOrchestrator._beforeSwap · twamm-in-unlock-skips-limit-order-ticks` · seen in 1/3 runs · NEW Pass 2 checked whether TWAMM virtual swaps trigger limit-order tick crossing.	**FIXED**	Both in-unlock and executor TWAMM swaps execute `poolManager.swap` without setting `TransientContext.isInternalHookExecution`, so `_afterSwap -> _runAfterSwapPipeline` always runs `_processCrossedLimitOrderTicks` on TWAMM swaps (HookRigOrchestrator.sol:436-438).
S5-F13	91	**`TWAMMLogic._distributeDirection` uses `Math.Rounding.Ceil` for `inShare`, causing `sum(swappedInputAmount)` to exceed `actualIn` or under-accounting `Floor` output shares** `TWAMMBlock._distributeDirection · shared-escrow-rounding-shortfall` · seen in 3/3 runs · NEW All 3 passes noted that `Ceil` rounding on `inShare` could cause `sum(swappedInputAmount)` across partial fills to exceed `actualIn`, over-debiting makers.	**FIXED**	`TWAMMLogic._distributeDirection` tracks running `remUnswapped` and `remActualIn` with ceiling division (`Math.Rounding.Ceil`), guaranteeing `sum(inShare) == min(actualIn, totalUnswapped)` and `inShare <= unswapped` for every maker so neither over-debiting nor escrow shortfall (`GL-04`) can occur (TWAMMBlock.sol:534-561).
S5-F14	90	**`LaunchpadFactory.launchSteppedTranche` caps `graduationQuoteTarget` at `tokensPerTranche * basePriceQ96 >> 96`, restricting multi-tranche graduation targets** `LaunchpadFactory.launchSteppedTranche · single-tranche-graduation-cap` · seen in 2/3 runs · KNOWN Single-tranche graduation target bound in `LaunchpadFactory.launchSteppedTranche` (`S2-F10`, `S1-F25`).	**BY-DESIGN**	Intentional single-tranche graduation bound required by invariant property `GL-15` and `SP-13` so single-price order evaluation never crosses multiple tranches (`docs/05`).
S5-F15	90	**`SteppedTrancheLaunch.claimStalledSurchargeRefund` requires `realQuoteRaised == 0`, which can be permanently blocked by 1-wei floor rounding dust in `sell()`** `SteppedTrancheLaunch.claimStalledSurchargeRefund · stalled-surcharge-refund-locked-by-dust` · seen in 2/3 runs · NEW Pass 2 & 3 found that `sell()` floors `proRataQuoteOut`, which can leave 1-wei rounding dust in `realQuoteRaised` when `curveTokensSold` reaches `0`.	**FIXED**	`SteppedTrancheLaunch.claimStalledSurchargeRefund` checks `curveTokensSold > 0` without requiring `realQuoteRaised == 0` and refunds full `buyerSurcharges[msg.sender]` even when tokens were bought for a separate `recipient` (SteppedTrancheLaunch.sol:245-268).
S5-F16	90	**`TWAMMLogic.submitOrder` allows 365-day orders at `MIN_TWAMM_ORDER_AMOUNT = 1e15` (`sellRate = 31,709,791 wei/s`), letting an attacker squat all 256 active order slots for 0.256 ETH** `TWAMMBlock.submitOrder · active-order-cap-squatting` · seen in 2/3 runs · KNOWN Economic cost of filling `MAX_ACTIVE_TWAMM_ORDERS = 256` slots at `MIN_TWAMM_ORDER_AMOUNT = 1e15`.	**BY-DESIGN**	`MIN_TWAMM_ORDER_AMOUNT = 1e15` is calibrated to allow legitimate orders across 6-to-18 decimal tokens while `MAX_ACTIVE_TWAMM_ORDERS = 256` bounds gas (`docs/04`).
S5-F17	89	**Unchecked `quoteToSeed * tokensToSeed` in `VirtualXYKLaunch.unlockCallback` and `SteppedTrancheLaunch.unlockCallback` overflows `uint256` when product exceeds `2^256 - 1`** `VirtualXYKLaunch.unlockCallback · graduation-liquidity-mul-overflow` · seen in 2/3 runs · NEW Pass 2 & 3 found that `Math.sqrt(quoteToSeed * tokensToSeed)` overflows if `quoteToSeed * tokensToSeed >= 2^256` on high-supply/high-quote launches.	**FIXED**	Both `VirtualXYKLaunch.unlockCallback` and `SteppedTrancheLaunch.unlockCallback` now compute `Math.sqrt(quoteToSeed) * Math.sqrt(tokensToSeed)` when either operand exceeds `type(uint128).max` (VirtualXYKLaunch.sol:250-253, SteppedTrancheLaunch.sol:337-340).

ID	Conf.	Finding	Status	Current evidence
S5-F18	88	`HookRigOrchestrator._runAfterSwapPipeline` **runs** `_checkAndAdvanceCircuitBreaker` **on TWAMM virtual swaps, allowing large accrued TWAMM volume to trip the circuit breaker and lock** `claimTWAMMOrder` **/** `cancelTWAMMOrder` `HookRigOrchestrator._runAfterSwapPipeline · twamm-circuit-breaker-pool-lock` · seen in 2/3 runs · NEW Pass 2 & 3 found that `_checkAndAdvanceCircuitBreaker` was not gated by `!isTWAMMInternal` in `_runAfterSwapPipeline`.	FIXED	HookRigOrchestrator._runAfterSwapPipeline now gates both `_enforceMEVAfterSwap` and `_checkAndAdvanceCircuitBreaker` inside `if (!isTWAMMInternal)` (HookRigOrchestrator.sol:384-393).
S5-F19	88	**In-unlock TWAMM virtual swaps in** `_beforeSwap` **execute** `_resolveBeforeSwapFee` **a second time and overwrite** `PreSwapSnapshot` **before the user swap runs** `HookRigOrchestrator._beforeSwap · twamm-double-dynamic-fee-observation` · seen in 1/3 runs · NEW Pass 3 found that TWAMM internal swaps overwrote `TransientContext.storePreSwapSnapshot` and consumed `loadAndClearPreSwapSnapshot`.	FIXED	HookRigOrchestrator._beforeSwap and `_runAfterSwapPipeline` now gate `storePreSwapSnapshot` and `loadAndClearPreSwapSnapshot` on `!isTWAMMInternal` (HookRigOrchestrator.sol:344-346,384-386).
S5-F20	88	`LimitOrderSwapExecutor` **swaps filled buckets in a single market order using the last claimer's** `minAmountOut`**, exposing earlier makers in the same bucket to sandwich slippage** `LimitOrdersBlock.claimOrder · shared-bucket-sandwich-and-slippage` · seen in 1/3 runs · KNOWN First claimer in a filled bucket triggers the batch market swap for that bucket (`S1-F21`, `TODO-20`).	BY-DESIGN	Tick-bucketed trigger orders execute as a single bucket swap on first claim and accept `minAmountOut` on `claimLimitOrder(key, orderId, minAmountOut)` (`docs/03`, `docs/04`).
S5-F21	87	`AccessControlLogic.enforceTwoSidedLiquidity` **checks strict** `< currentTick <` **inequality, allowing single-sided liquidity in the active tick spacing bin** `[alignedTick, alignedTick + tickSpacing]` `AccessControlBlock.enforceTwoSidedLiquidity · active-bin-single-sided-liquidity-bypass` · seen in 1/3 runs · KNOWN A range straddling `currentTick` (`tickLower < currentTick < tickUpper`) is always two-sided at exact `sqrtPriceX96` unless `sqrtPriceX96` sits strictly on a boundary tick.	BY-DESIGN	Strict `params.tickLower < currentTick && params.tickUpper > currentTick` guarantees both token0 and token1 are required at `currentTick` (`AccessControlBlock.sol:140-150`).
S5-F22	86	`SteppedTrancheLaunch.buy` **evaluates the entire purchase at the entry tranche's** `currentMarginalPriceQ96()` **without splitting across tranche boundaries** `SteppedTrancheLaunch.buy · multi-tranche-single-price-execution` · seen in 2/3 runs · KNOWN Known design item (`S2-F13`, `S3-F29`, `SP-13`): `LaunchpadFactory` bounds `graduationQuoteTarget` to a single tranche so orders never cross multiple tranches.	BY-DESIGN	Verified by `SP-13` in `test/fizz/Properties.sol` and bounded by `LaunchpadFactory.launchSteppedTranche` (`docs/05`).
S5-F23	85	`SteppedTrancheLaunch.sell` **caps redemption at** `min(basePriceQ96, proRata)` **and burns** `cumulativeTokensBought` **allowance on round-trip trades** `SteppedTrancheLaunch.sell · asymmetric-sell-redemption-and-cap-burn` · seen in 2/3 runs · KNOWN Known design item (`S1-F27`, `SP-05`, `SP-13`): `min(basePriceQ96, proRata)` guarantees `SP-05` (no profitable buy->sell round trip) and curve solvency (`GL-14`).	BY-DESIGN	Required by `SP-05`, `SP-13`, and `GL-14` in `test/fizz/Properties.sol` (`docs/05`).

ID	Conf.	Finding	Status	Current evidence
S5-F24	85	`VestingSellCapLogic.enforce` **snapshots** `epochStartBalance` **lazily on first sell and allows multi-wallet splitting across EOAs** `VestingSellCapBlock._enforceVestingSellCap · multi-wallet-epoch-cap-bypass` · seen in 2/3 runs · KNOWN Known design limit (`S2-F17`, `TODO-21`): pool-level Uniswap v4 hooks only observe swap callbacks, not off-pool `ERC20.transfer` calls.	BY-DESIGN	Outflows lower the active epoch's effective start balance (`VestingSellCapBlock.sol:63-76`), and off-pool EOA splitting is documented in `docs/04` §3.7.
S5-F25	85	`AutoBuybackLogic.processWaterfallBuyback` **executes an unsplit market swap with** `MIN_SQRT_PRICE + 1` **/** `MAX_SQRT_PRICE - 1` **when** `reserve >= minBuybackAmount` `AutoBuybackBlock.processWaterfallBuyback · predictable-market-buyback-sandwich` · seen in 2/3 runs · KNOWN Known design behaviour (`TODO-28`): on-chain threshold buybacks execute at market when the floor tick is breached and `reserve >= minBuybackAmount`.	BY-DESIGN	Buyback proceeds are burned (`0xdEaD`) and LP donations are delayed until a block without liquidity additions to block JIT capture (`SP-24`, `docs/04`).
S5-F26	84	`CircuitBreakerLogic.checkAndAdvance` **clamps new window anchor ticks to** `prevAnchor ± maxWindowTickDeviation` **instead of resetting to** `postSwapTick` **after cooldown** `CircuitBreakerBlock._checkAndAdvanceCircuitBreaker · cascading-circuit-breaker-retrigger` · seen in 2/3 runs · KNOWN Known design item (`S1-F07`, `S3-F68`, `TODO-28`): clamping window rollover anchor prevents an attacker fromratcheting the anchor to a manipulated tick in a single window.	BY-DESIGN	Documented in `CircuitBreakerBlock.sol` NatSpec and `docs/04`.
S5-F27	83	`OracleLogic.update` **clamps per-second tick movement to** `MAX_ABS_TICK_MOVE_PER_SEC = 100`**, causing truncated TWAP to lag during genuine >1%/sec price discovery** `OracleBlock._updateOracleCheckpoint · truncated-twap-step-lag` · seen in 1/3 runs · KNOWN Truncated geometric-mean TWAP oracle design (`100 ticks/sec` clamp) prevents single-block flash-loan TWAP manipulation (`GL-19`).	BY-DESIGN	Required by `GL-19` (`property_GL19_oracleBounds`) and documented in `OracleBlock.sol` and `docs/04`.
S5-F28	82	`LimitOrdersLogic.placeOrder` **re-opens a filled bucket when** `currentTick` **has recrossed** `targetTick`**, advancing** `bucket.epoch` **while prior-epoch makers have not yet claimed** `LimitOrdersBlock.placeOrder · multi-epoch-shared-escrow-contention` · seen in 1/3 runs · NEW Pass 1 noted that re-opening a bucket while earlier filled orders remain unclaimed required separating `totalInputAmount` from `epochInputAmount`.	FIXED	LimitOrdersBlock separates `totalInputAmount` and `epochInputAmount` and snapshots `epochInputAmount` at claim swap execution (LimitOrdersBlock.sol:31-35,325-336).
S5-F29	82	`ArbRecaptureLogic.computeRecapture` **keeps** `referenceTick` **anchored across sub-threshold swaps and captures 50% of excess spread as an** `afterSwapReturnDelta` **cut** `ArbRecaptureBlock.computeRecapture · reference-tick-anchoring-and-cut-cap` · seen in 1/3 runs · KNOWN Known design item (`S2-F03`, `TODO-28`): keeping `referenceTick` anchored across sub-threshold swaps prevents split-swap evasion.	BY-DESIGN	Documented in `ArbRecaptureBlock.sol` NatSpec and `docs/04`.

ID	Conf.	Finding	Status	Current evidence
S5-F30	80	`MEVProtectionLogic.enforceAfterSwap` **locks opposite-direction swaps for the remainder of the block whenever a swap moves the tick by** `>= max(maxTickDeltaPerSwap / 20, 2)` `MEVProtectionBlock._enforceMEVAfterSwap · intra-block-direction-lock-griefing` · seen in 2/3 runs · KNOWN Known design item (`S2-F27`, `S3-F61`, `S3-F62`, `TODO-28`): intra-block opposite-direction lock after a non-dust swap is the core anti-sandwich mechanism.	BY-DESIGN	Dust swaps (`< dirThreshold`) are exempted (`test_Fix_MEVDustSwapDoesNotSetBlockDirection`), and threshold lock is documented in `docs/04`.
S5-F31	80	`KingOfThePoolLogic.processBuy` **crowns a buyer whose nominal pot fee is** `> 0` **even when earlier blocks in the waterfall already consumed the 25%** `MAX_TOTAL_RETURN_DELTA_BPS` **cap** `KingOfThePoolBlock.processBuy · zero-contribution-king-crowning` · seen in 1/3 runs · KNOWN Known design item (`S3-F44`, `TODO-28`): allows the King pot to remain contestable when seeded in quote currency or when earlier blocks fill the 25% cap.	BY-DESIGN	Tested in `test_Fix_KingSeedInOtherCurrencyStillContestable` and `test_Fix_KingRequiresPotFee` (`KingOfThePoolBlock.sol:162-170`).
S5-F32	79	`FeeAggregator._resolveBeforeSwapFee` **returns** `max(baseFeeEngine, antiSnipeFee)` **rather than summing them, while** `DynamicFee` **updates** `rollingVolatilityTicks` **only on new blocks when** `delta < ema` `FeeAggregator._resolveBeforeSwapFee · fee-precedence-and-volatility-floor` · seen in 1/3 runs · KNOWN Documented fee precedence waterfall (`docs/03`) and anti-wash-decay filter (`TODO-25`).	BY-DESIGN	Tested in `test_CustomStack_MultiFeeBlockPrecedenceWaterfall` and documented in `docs/03`.
S5-F33	78	`HookRigBase.registerPoolStack` **is permissionless (first-come-first-served) for any unconfigured standard** `PoolKey` **whose tokens have deployed bytecode** `HookRigBase.registerPoolStack · permissionless-first-come-pool-registration` · seen in 1/3 runs · KNOWN Known design item (`S2-F24`, `S3-F41`, `TODO-23`): `PoolKey` includes `tickSpacing`, `fee`, and `hooks`, so creators choose their own `PoolKey`.	BY-DESIGN	Undeployed tokens are rejected (`TokenNotDeployed`) and initialization is restricted to `poolAdmin` or `owner` (`docs/04`).
S5-F34	76	`VirtualXYKLaunch._graduateToV4` **and** `SteppedTrancheLaunch._graduateToV4` **lock permanent full-range liquidity inside** `PoolManager` **owned by the curve contract with no LP withdrawal function** `VirtualXYKLaunch._graduateToV4 · permanently-locked-graduation-lp` · seen in 1/3 runs · KNOWN Known design item (`S4-F22`, `S4-F30`, `TODO-21`): Pump.fun-style bonding curves permanently lock graduation LP so the creator can never rug liquidity.	BY-DESIGN	Intentional anti-rug mechanism documented in `docs/04` §3.7 and `docs/05`.
S5-L01	lead	`TWAMMSwapExecutor.unlockCallback` **settles input tokens from** `address(this)` **and refunds leftover input to** `hook` `TWAMMBlock.unlockCallback · twamm-executor-residual-routing` · seen in 3-pass runs · LEAD Verified that `TWAMMSwapExecutor` only holds tokens transiently inside `executeVirtualSwap` and returns all `leftover` input and `actualOut` to `hook` in the same transaction.	BY-DESIGN	Verified safe in `TWAMMBlock.sol:40-98`.

ID	Conf.	Finding	Status	Current evidence
S5-L02	lead	`NativeTickLaunch.unlockCallback` **burns unseeded residual project tokens to** `0xdEaD` **without adjusting** `totalSupply()` `NativeTickLaunch.unlockCallback · dead-address-residual-burn` · seen in 3-pass runs · LEAD `0xdEaD` is the canonical burn sink used across `AutoBurnBlock`, `VirtualXYKLaunch`, `SteppedTrancheLaunch`, and `NativeTickLaunch` (`GL-24`).	BY-DESIGN	Consistent with `GL-24` (`property_GL24_autoBurnLedger`) in `test/fizz/Properties.sol`.
S5-L03	lead	`LaunchpadVestingWallet.release` **blocks ERC-20 releases before** `graduated() == true` **or** `block.timestamp >= end()`**, but** `release()` **for native ETH is inherited** `LaunchpadFactory.release · native-eth-vesting-release` · seen in 3-pass runs · LEAD `LaunchpadVestingWallet` only receives `HookRigToken` ERC-20 allocations from `LaunchpadFactory._deployTokenAndEscrow`; it never holds native ETH.	BY-DESIGN	Verified in `LaunchpadFactory.sol:29-66`.
S5-L04	lead	`ReferralSplitBlock.registerReferrer` **is one-time per** `(poolId, msg.sender)` **and cannot be rotated if a referrer address is compromised** `ReferralSplitBlock.registerReferrer · immutable-referrer-binding` · seen in 3-pass runs · LEAD First-touch immutable referral attribution prevents wash-rotation of referrers (`test_Block20_ReferralSplit_FirstTouchAndAntiSelfReferral`).	BY-DESIGN	Intentional first-touch referral design (`ReferralSplitBlock.sol`).
S5-L05	lead	`SwapRewardsBlock._accrueSwapRewards` **credits points to** `ctx.trader`**, which equals** `sender` **(the router) when swapping through an untrusted router without** `hookData` `SwapRewardsBlock._accrueSwapRewards · untrusted-router-points-attribution` · seen in 3-pass runs · LEAD Non-transferable loyalty points (`accruedSwapPoints`) accrue to the resolved trader identity (`docs/04`).	BY-DESIGN	Documented in `SwapRewardsBlock.sol` and `docs/04`.
S5-L06	lead	`BuySellTaxLogic.initRulesAndRewardsBlocks` **allows** `decayOnly = false` **when configured by** `poolAdmin` **at registration** `BuySellTaxBlock.initRulesAndRewardsBlocks · admin-mutable-tax-within-cap` · seen in 3-pass runs · LEAD Standard pools allow `poolAdmin` to choose `decayOnly = true` or `false` within `MAX_TAX_BPS = 1000` and `2500 bps` aggregate budget, whereas `LaunchpadFactory` forces `decayOnly = true`.	BY-DESIGN	Verified by `test_Fix_LaunchRejectsUnsafeStacksAndHooks` and `test_Fix_TaxCannotIncreaseWithoutDecayOnly`.
S5-L07	lead	`MarketHoursFeeLogic.computeFee` **uses** `(block.timestamp / 3600) % 24` **(UTC hours) without daylight-saving adjustments** `MarketHoursFeeBlock.computeFee · utc-hour-no-dst-shift` · seen in 3-pass runs · LEAD On-chain EVM schedules use deterministic UTC hours (`openHourUtc`, `closeHourUtc`) rather than off-chain timezone/DST rules.	BY-DESIGN	Documented in `MarketHoursFeeBlock.sol` and `docs/01`.

ID	Conf.	Finding	Status	Current evidence
S5-L08	lead	`ManagedFeeBlock.setManagedPoolFee` **allows** `feeManager` **to update the static/dynamic LP fee up to** `hardCapFee` **(<=** `50_000`**) immediately without timelock** `ManagedFeeBlock.setManagedPoolFee · immediate-managed-fee-update` · seen in 3-pass runs · LEAD `ManagedFeeBlock` is classified as Trust Level: Medium (`docs/01`) and is bounded by the immutable per-pool `hardCapFee <= 50_000` (5%).	BY-DESIGN	Tested in `test_Block02_ManagedFee_UpdateAndHardCap` (`ManagedFeeBlock.sol`).
S5-L09	lead	`AutoBurnLogic._burnTokens` **falls back to transferring tokens to** `0xdEaD` **if** `IBurnableERC20(token).burn(amount)` **reverts** `AutoBurnBlock._burnTokens · burn-fallback-to-dead-address` · seen in 3-pass runs · LEAD Try/catch fallback to `0xdEaD` guarantees non-burnable or paused-burn tokens never brick swaps (`test_Block10_AutoBurn_BurnCallAndDeadAddressModes`).	BY-DESIGN	Verified by `GL-24` and `test_Block10_AutoBurn_BurnCallAndDeadAddressModes`.
S5-L10	lead	`KingOfThePoolLogic.validateClaim` **caps payout at** `manager.balanceOf(address(this), currency.toId())` **if ERC-6909 backing is ever below** `settled.amount` `KingOfThePoolBlock.validateClaim · defensive-erc6909-balance-cap` · seen in 3-pass runs · LEAD Defensive cap ensures `claimKingPot` never reverts with ERC-6909 underflow while `GL-02` proves 100% ERC-6909 backing across all pools.	BY-DESIGN	Verified by `GL-02` (`property_GL02_fullBacking`) in `test/fizz/Properties.sol`.
S5-L11	lead	`AutoBuybackLogic.flushPendingDonations` **skips** `manager.donate` **when active in-range liquidity is zero and retains the pending donation for a future swap** `AutoBuybackBlock.flushPendingDonations · zero-liquidity-donation-retention` · seen in 3-pass runs · LEAD Retaining `pendingLPDonations` when in-range liquidity is `0` prevents wasting rewards on empty ticks (`test_AttackVector04_ZeroLiquidityDonateFallbackDoesNotRevert`).	BY-DESIGN	Verified in `AutoBuybackBlock.sol:145-175`.
S5-L12	lead	`HookRigBase.withdrawClaim` **transfers full** `poolClaims[poolId][currency][msg.sender]` **and zeroes storage before burning ERC-6909 claims and calling** `manager.take` `HookRigBase.withdrawClaim · pull-claim-ceI-ordering` · seen in 3-pass runs · LEAD Follows strict Checks-Effects-Interactions (`nonReentrant`) and is verified by `SP-01` (`property_SP01_withdrawClaimExact`).	BY-DESIGN	Verified in `HookRigBase.sol:124-137`.
S5-L13	lead	`WalletCapLogic.enforceAfterSwap` **reads** `IERC20(token).totalSupply()`**, which decreases when** `AutoBurnBlock` **burns tokens via** `ERC20Burnable.burn` `WalletCapBlock._enforceWalletCapAfterSwap · burn-reduces-dynamic-wallet-cap` · seen in 3-pass runs · LEAD Dynamic `totalSupply()` check applies only to subsequent buys while `sunsetBlocks` is active (`SP-25`).	BY-DESIGN	Verified by `SP-25` (`property_SP25_walletCapNoBypass`) in `test/fizz/Properties.sol`.

ID	Conf.	Finding	Status	Current evidence
S5-L14	lead	`AccessControlLogic.verifyAccess` **caches** `allowlisted[poolId][user] = true` **after the first valid Merkle proof verification** `AccessControlBlock._verifyAccess · merkle-proof-allowlist-caching` · seen in 3-pass runs · LEAD `setMerkleRoot` increments storage or `setAllowlistAccount(poolId, user, false)` can revoke cached accounts if needed.	**BY-DESIGN**	Saves gas on repeat swaps by the same Merkle-verified user (`AccessControlBlock.sol:190-210`).
S5-L15	lead	`TransientContext` **uses deterministic** `keccak256` **EIP-1153 slots per** `PoolId` **without clearing across distinct** `PoolManager.unlock` **calls in the same transaction if** `afterSwap` **is not enabled** `TransientContext.loadAndClearPreSwapSnapshot · transient-slot-lifecycle` · seen in 3-pass runs · LEAD `HookPermissionMasks.permissionsForBitmap` always enables `afterSwap` whenever `beforeSwap` is enabled (`perms.afterSwap = perms.beforeSwap || ...`), so snapshots are always cleared.	**FIXED**	Enforced in `HookPermissionMasks.sol:207` and `TransientContext.sol:52-55`.
S5-L16	lead	`LaunchpadFactory._validateStackSafety` **rejects** `BIT_ACCESS_CONTROL`**,** `BIT_MANAGED_FEE`**, and** `BIT_CIRCUIT_BREAKER` **on launchpad pools** `LaunchpadFactory._validateStackSafety · launchpad-restricted-blocks` · seen in 3-pass runs · LEAD Prevents malicious token creators from bricking trading on graduated launchpad pools via admin allowlists, 5% managed fees, or hair-trigger circuit breakers.	**BY-DESIGN**	Tested in `test_Fix_LaunchRejectsUnsafeStacksAndHooks` (`LaunchpadFactory.sol:220-256`).
S5-L17	lead	`PoolTokenDir.resolveSwapContext` **treats** `exactInput` **and** `exactOutput` **symmetrically by normalizing** `grossUnspecifiedAmount` **and** `settledSpecifiedAmount` `PoolTokenDir.resolveSwapContext · exact-output-cut-semantics` · seen in 3-pass runs · LEAD Exact-output cuts increase the amount the swapper must pay (`hookDeltaUnspecified > 0`), verified by `SP-11` (`property_SP11_exactOutputCutCeil`).	**BY-DESIGN**	Verified by `SP-11` and `test_AttackVector05_ExactInAndExactOut_Token0AndToken1Polarity`.
S5-L18	lead	`HoodiSwapRouter.swap` **refunds excess** `msg.value` **to** `msg.sender` **via low-level** `.call{value: ...}("")` **inside** `unlockCallback` `HoodiIntegration.unlockCallback · router-native-refund-call` · seen in 3-pass runs · LEAD `HoodiSwapRouter` is a stateless test/deployment script helper (`script/HoodiIntegration.s.sol`) that caps native settlement at `cb.msgValue` (`S4-L04`).	**FIXED**	Verified in `script/HoodiIntegration.s.sol:69-85`.

10. Medusa & Echidna Fuzzing — 2026-10-15 – 2026-10-16 (3b46f47+)

Stateful property-based fuzzing across 7 campaigns (2 initial 600s Medusa discovery passes, 1 Echidna 1800s campaign, 2 post-`/fizz-sync` full-route Medusa passes, 1 deep 250,641-call Echidna campaign, and 1 deep 250,000-call Medusa campaign) over `3b46f47+` (56 properties · 98 routes · 500k+ deep calls). Remediation: working tree (all 21 Medusa & Echidna findings remediated + 23 FoundryTester repros). 21 findings reproduced deterministically in `test/fizz/FoundryTester.sol`; current status: 21 fixed.

ID	Conf.	Finding	Status	Current evidence
FZ-01	100	**OracleLogic.updateCheckpoint int24 truncation overflows after ~23.3h idle and freezes swaps and LP exits** `OracleBlock.updateCheckpoint · GL-19 / SP-21` · Medusa Pass 1 & 2 · GL-19 / SP-21 `int24 maxChange = int24(int256(uint256(elapsed)) * 100)` truncates once `elapsed > 83,886 s`, pushing the clamped tick below `MIN_TICK` or panicking with 0x11 on every swap and LP exit.	**FIXED**	`_clampTick` computes `maxChange` in `int256` and clamps within `[MIN_TICK, MAX_TICK]` before casting to `int24` (OracleBlock.sol:112-127). Repros: `test_repro_GL19_oracleBounds`, `test_repro_SP21_lpExitAfterHold`.
FZ-02	100	**KingOfThePoolBlock currency mixing between launchpad quote seed and project-token buy cuts leaves shared ERC-6909 claims under-backed** `KingOfThePoolBlock.processBuy & seedPotFromLaunchCurve · GL-02 / GL-03` · Medusa Pass 2 · GL-02 / GL-03 Seeding `currency0` into Round 1 and then overwriting `potCurrency` with the project token on the next buy mixed currencies in one pot, leaving shared ERC-6909 claims insolvent across pools.	**FIXED**	`KingOfThePoolBlock` locks `potCurrency` while `currentPotAmount > 0`, adds cuts only when `potCurrency == cutCurrency`, and caps claims at available ERC-6909 backing. Repros: `test_repro_GL02_fullBacking`, `test_repro_GL03_kingPotCurrencyPurity`.
FZ-03	100	**LimitOrdersLogic.placeOrder records native ETH limit orders without msg.value escrow, draining shared hook ETH on cancel/claim** `LimitOrdersBlock.placeOrder · SP-03` · Medusa Pass 1 · SP-03 Placing a native ETH limit order skipped ETH transfer on a non-payable entrypoint while recording `inputAmount`, allowing `cancelLimitOrder` / `claimLimitOrder` to drain ETH held by the hook.	**FIXED**	`placeOrder` is payable, enforces `msg.value == inputAmount` for native currency (`msg.value == 0` for ERC-20), and isolates per-pool escrow in `limitOrderPoolEscrow` (LimitOrdersBlock.sol:193-205). Repro: `test_repro_SP03_nativeLimitOrderEscrow`.
FZ-04	100	**HookRigBase._resolveUser trusts hookData[0:32] from untrusted routers, allowing AccessControl allowlist spoofing** `HookRigBase._resolveUser & AccessControlBlock.resolveUser · SP-15` · Medusa Pass 1 · SP-15 Passing a spoofed allowlisted address in `hookData[0:32]` through an untrusted router allowed non-allowlisted callers to trade on `POOL_PRIVATE`.	**FIXED**	`AccessControlLogic.resolveUser` never decodes `hookData`; it queries `IMsgSender(router).msgSender()` only for trusted routers and falls back to `routerSender` (AccessControlBlock.sol:238-251). Repro: `test_repro_SP15_accessControlSpoof`.
FZ-05	95	**VirtualXYKLaunch and SteppedTrancheLaunch graduate at average reserve ratio instead of final marginal curve price** `VirtualXYKLaunch._graduateToV4 & SteppedTrancheLaunch._graduateToV4 · GL-11` · Medusa Pass 1 · GL-11 Seeding the Uniswap v4 pool at `lpTokenReserve / quoteToSeed` instead of the curve's final marginal price created an immediate arbitrage gap at graduation.	**FIXED**	Both curves compute initial `sqrtPriceX96` from the final marginal curve price (`lastFillPriceQ96` on SteppedTranche) and scale `tokensToSeed` / `quoteToSeed` proportionally. Repro: `test_repro_GL11_graduationPriceContinuity`.

ID	Conf.	Finding	Status	Current evidence
FZ-06	98	**TWAMMLogic sell rates never expire, streaming virtual orders forever past their submitted volume** `TWAMMBlock.executePendingWithSwap · GL-17 · Medusa Pass 1 · GL-17` Increasing `sellRate0For1` / `sellRate1For0` with += without subtracting expired order rates (`expirationTimestamp`) caused virtual orders to stream indefinitely.	FIXED	`TWAMMLogic` tracks `order.rateRemoved` and subtracts `sellRatePerSecond` once `block.timestamp >= order.expirationTimestamp` (TWAMMBlock.sol:404-415). Repro: `test_repro_GL17_twammBounded`.
FZ-07	90	**AfterSwapDeltaAggregator._boundedCut rounds exact-output fee cuts down (Floor) instead of up (Ceil)** `AfterSwapDeltaAggregator._boundedCut · SP-11 · Medusa Pass 1 · SP-11` Using `Math.Rounding.Floor` on exact-output swap cuts under-charged the added input cost by 1 wei relative to the Bunni V2 Ceil rounding invariant.	FIXED	`_boundedCut` uses `Math.Rounding.Ceil` when `!ctx.isExactInput` so exact-output cuts round in favor of the pool/hook (AfterSwapDeltaAggregator.sol:234-245). Repro: `test_repro_SP11_exactOutputCutCeil`.
FZ-08	95	**HookRigOrchestrator.registerPoolStack allows third-party retro-registration on an already-initialized live pool** `HookRigOrchestrator.registerPoolStack & _verifyPoolInit · SP-19 · Medusa Pass 1 · SP-19` An unconfigured pool that was initialized on `PoolManager` could later have its block stack registered and locked by an arbitrary third party.	FIXED	`registerPoolStack` reverts if `sqrtPriceX96 != 0` (`PoolAlreadyInitialized`) or token bytecode is missing (`TokenNotDeployed`), and `_verifyPoolInit` blocks unconfigured `initialize` (AccessControlBlock.sol:74-105). Repro: `test_repro_SP19_registerOnLivePool`.
FZ-09	98	**Unescrowed uint128-max TWAMM order overflows sellRate accumulator and bricks all subsequent order submissions** `TWAMMBlock.submitOrder · SP-30 · Medusa Pass 1 · SP-30` Submitting a free `totalAmount = type(uint128).max` order with `duration = 1` overflowed `sellRate` and caused all subsequent honest TWAMM submissions to revert.	FIXED	`submitOrder` requires real token/ETH escrow transfer with balance-delta check, `MIN_TWAMM_ORDER_AMOUNT = 1e15`, and `MAX_ACTIVE_TWAMM_ORDERS = 256` (TWAMMBlock.sol:106-107,200-224). Repro: `test_repro_SP30_accumulatorGrief`.
FZ-10	95	**LaunchpadFactory lacks curveTokenAllocation + lpTokenReserve <= totalSupply - creatorVesting validation** `LaunchpadFactory._deployTokenAndEscrow · GL-23 · Medusa Pass 1 · GL-23` Launching with creator vesting without verifying that `totalSupply - creatorVesting` covers `curveTokenAllocation + lpTokenReserve` left the curve under-funded for graduation.	FIXED	`LaunchpadFactory` validates `curveTokenAllocation + lpTokenReserve <= availableForCurve` on both VirtualXYK and SteppedTranche launches (LaunchpadFactory.sol:80-150). Repro: `test_repro_GL23_curveTokenAccounting`.
FZ-11	95	**SteppedTrancheLaunch.buy caps tokensOut at remaining allocation without refunding unused quote input** `SteppedTrancheLaunch.buy · SP-27 · Medusa Pass 1 · SP-27` When a buy crossed `curveTokenAllocation`, `tokensOut` was capped at `remainingTokens` while the buyer was still charged the full `quoteAmountIn`.	FIXED	When `tokensOut > remainingTokens`, `SteppedTrancheLaunch.buy` recomputes `netQuoteForTokens` (Ceil) and `surcharge` from the capped `remainingTokens` and only pulls `quoteUsed` (SteppedTrancheLaunch.sol:132-160). Repro: `test_repro_SP27_trancheChargedUndelivered`.
FZ-12	92	**Single SteppedTrancheLaunch.buy order skips higher tranche prices when graduationQuoteTarget spans multiple tranches** `SteppedTrancheLaunch.buy & LaunchpadFactory.launchSteppedTranche · GL-15 · Medusa Pass 2 · GL-15` Pricing an entire buy at the entry tranche's `currentMarginalPriceQ96()` allowed a large buy to pay `basePriceQ96` across multiple tranches.	FIXED	`LaunchpadFactory.launchSteppedTranche` caps `graduationQuoteTarget` at a single tranche's base cost (`tokensPerTranche * basePriceQ96 >> 96`) so single-price evaluation never crosses multiple tranches (LaunchpadFactory.sol:124-135). Repro: `test_repro_GL15_trancheScheduleFloor`.

ID	Conf.	Finding	Status	Current evidence
FZ-13	95	**Filled limit orders return original input currency instead of executing a swap into output currency on claim** `LimitOrdersBlock.claimOrder · SP-28` · Medusa Pass 1 & 2 · SP-28 `_swapFilledOrderOnClaim` returned `false`, so `claimLimitOrder` refunded the maker's input token rather than converting filled buckets to the output token.	**FIXED**	`_swapFilledOrderOnClaim` is enabled (`true`) and `claimOrder` executes the bucket fill swap via `LimitOrderSwapExecutor` with `minAmountOut` slippage protection (LimitOrdersBlock.sol:290-340,400-402). Repro: `test_repro_SP28_filledOrderPaysOutput`.
FZ-14	92	**Unsold curve allocation and direct token donations remain permanently stranded in graduated curve contracts** `VirtualXYKLaunch._graduateToV4 & SteppedTrancheLaunch._graduateToV4 · GL-26` · Medusa Pass 1 · GL-26 After seeding `tokensToSeed` into `PoolManager`, any unsold curve allocation or donated project tokens remained locked inside the curve contract forever.	**FIXED**	Both curves sweep all residual project tokens above `tokensToSeed` to `DEAD_ADDRESS (0xdEaD)` during graduation (VirtualXYKLaunch.sol:260-268, SteppedTrancheLaunch.sol:348-356). Repro: `test_repro_GL26_noStrandedCurveValue`.
FZ-15	95	**JITGuardBlock hold period bypassed by switching between honest hookData on addLiquidity and empty hookData on removeLiquidity** `JITGuardBlock.recordAddition & enforceOnRemove · SP-16` · Targeted Fuzz Probe · SP-16 Adding liquidity via a router with `hookData` recorded the hold under the actor's key, while removing in the same block with empty `hookData` checked the router's key and bypassed `minLPHoldBlocks`.	**FIXED**	`JITGuardBlock` tracks per-key `lpTrackedLiquidity` and falls back to `posKey` hold enforcement whenever `lpTrackedLiquidity[ownerKey] < removedLiq` (JITGuardBlock.sol:34-155). Repro: `test_repro_SP16_jitHoldIdentitySwitch`.
FZ-16	98	**Exact-output buy swaps pass quote currency into AutoBurnBlock instead of project token, corrupting the burn ledger** `AfterSwapDeltaAggregator._executeAfterSwapDeltaWaterfall · GL-24` · Medusa Pass 1 · GL-24 On exact-output buys, `unspecifiedCurrency` is the quote currency, causing `AutoBurnBlock` to burn buyer quote tokens instead of project tokens.	**FIXED**	Exact-output buys revert with `ExactOutputBuyNotSupported` when `BIT_AUTO_BURN` or `BIT_KING_OF_THE_POOL` is enabled, and `AutoBurn` requires `ctx.unspecifiedCurrency == ctx.projectCurrency` (AfterSwapDeltaAggregator.sol:60-65,113-116). Repro: `test_repro_GL02_fullBacking`.
FZ-17	96	**Echidna 9-call sequence exposes SteppedTrancheLaunch.sell redemption rounding & cumulativeTokensBought cap desync** `SteppedTrancheLaunch.sell · SP-13` · Echidna 1800s Campaign · SP-13 Interleaved buy/sell round-trips and dust cycles in Echidna surfaced redemption discrepancies unless `quoteOut` is capped at `min(baseRedemption, proRataCap, realQuoteRaised)`.	**FIXED**	`SteppedTrancheLaunch.sell` enforces `cumulativeTokensBought[msg.sender] >= tokenAmountIn` and caps `quoteOut` at `min(baseRedemption, proRataCap, realQuoteRaised)` (SteppedTrancheLaunch.sol:185-235). Repro: `test_repro_echidna_SP13_steppedTrancheSellModel`.
FZ-18	92	**Cross-currency queued LP donation flushed in beforeSwap interacts with opposite-direction afterSwapReturnDelta cut accounting** `AutoBuybackBlock.flushPendingDonations & AfterSwapDeltaAggregator · SP-10` · Medusa /fizz-sync Pass 1 · SP-10 A buy on `POOL_MULTI` queues a `currency1` LP donation (`pendingLPDonations`), and a subsequent sell flushes `currency1` ERC-6909 in `beforeSwap` while taking a `currency0` waterfall cut in `afterSwap`.	**FIXED**	Verified that queued LP donations flushed in `beforeSwap` (`dDon0`, `dDon1`) are isolated from the current swap's 25% `MAX_TOTAL_RETURN_DELTA_BPS` waterfall cut (Properties.sol:982-1007). Repro: `test_repro_SP10_crossCurrencyDonationFlush`.

ID	Conf.	Finding	Status	Current evidence
FZ-19	98	**SteppedTrancheLaunch.claimStalledSurchargeRefund permanently locks anti-snipe surcharges when recipient != msg.sender or tokens were transferred before sell** `SteppedTrancheLaunch.claimStalledSurchargeRefund` · SP-22 · Medusa /fizz-sync Pass 2 · SP-22 Even though `claimStalledSurchargeRefund` requires `curveTokensSold == 0` (all curve tokens returned) and sets `stalledRefundActive = true`, gating `refundAmount` on `cumulativeTokensSoldByBuyer[msg.sender] / cumulativeTokensBought[msg.sender]` caused a `ZeroAmount()` revert or partial lock whenever tokens were bought for a separate `recipient` or sold back from another wallet.	**FIXED**	`claimStalledSurchargeRefund` now refunds the caller's full `buyerSurcharges[msg.sender]` (capped at `totalSurchargeCollected`) once `curveTokensSold == 0` past the stall window (SteppedTrancheLaunch.sol:245-268). Repro: `test_repro_SP22_stalledSurchargeRecipientLock`.
FZ-20	98	**TWAMMLogic._distributeDirection independent floor rounding on partial fills under-debits swappedInputAmount, breaking GL-04 escrow solvency and bricking final claimTWAMMOrder / cancelTWAMMOrder** `TWAMMBlock._distributeDirection · GL-04` · Medusa /fizz-sync Pass 2 · GL-04 When a virtual TWAMM swap partially fills `totalUnswapped` (`actualIn < totalUnswapped` at price bounds or thin liquidity), computing `inShare = mulDiv(actualIn, unswapped, totalUnswapped, Floor)` independently across 2+ orders left `sum(inShare) < actualIn`. Total refundable input (`totalAmount - swappedInputAmount`) exceeded the hook's ERC-20 balance by up to `N - 1` wei, cau…	**FIXED**	`_distributeDirection` now tracks running `remUnswapped` and `remActualIn` using ceiling division (`Math.Rounding.Ceil` with `remUnswapped <= unswapped ? remActualIn : ...`), guaranteeing `sum(inShare) == min(actualIn, totalUnswapped)` and `inShare <= unswapped` (TWAMMBlock.sol:585-611). Repro: `test_repro_GL04_twammPartialFillRoundingInsolvency`.
FZ-21	100	**TWAMMLogic.executePendingWithSwap out-of-range getLiquidity == 0 skip allows atomic 2-leg sandwich to capture frozen TWAMM backlog for instant round-trip profit** `TWAMMBlock.executePendingWithSwap & _swapVirtualInput · SP-04` · Medusa 250k Deep Campaign · SP-04 `TWAMMLogic.executePendingWithSwap` previously checked `if (StateLibrary.getLiquidity(poolManager, poolId) == 0) return;`. Because `StateLibrary.getLiquidity` only checks in-range liquidity at the current tick, whenever an earlier swap pushed the pool tick outside the active LP range (`[-600, 600]`), `executePendingWithSwap` skipped pending TWAMM virtual swaps that would cross …	**FIXED**	Removed the `getLiquidity == 0` early return, gated virtual swaps on `sqrtPriceX96` boundaries (> `MIN_SQRT_PRICE + 1` / < `MAX_SQRT_PRICE - 1`) with `ZeroVirtualSwapOutput` rollback via `TWAMMSwapExecutor.trySwapInUnlock`, and re-executed residual opposing backlog in `_executeAmmSwaps` (TWAMMBlock.sol:480-544,647-679). Repro: `test_repro_SP04_twammOutOfRangeBacklogSandwich`.

11. Test Evidence, Fuzzing Coverage & Disclaimer

Final state verified on 2026-10-16: `forge test` passes all 100 tests across all 7 suites (21 in `BlocksUnitTest`, 21 in `AuditFixesTest`, 21 in `PresetsAndSecurityVectorsTest`, 5 in `LaunchpadSuiteTest`, 4 in `HookRigInvariantTest`, 25 in `FoundryTester` including all 23 Medusa/Echidna `test_repro_*` regression tests, and 3 in `HoodiForkIntegrationTest`, including the exhaustive verification over all 2,843 valid Tier-3 block combinations). `medusa fuzz --config medusa.json --test-limit 250000 --seq-len 150` passes `98 / 98` routes (`0` failures) and `echidna . --contract FuzzTester --config echidna.yaml` (`testLimit: 250000`, `seqLen: 150`) passes `99 / 99` targets (`0` failures) across the 56 formal properties (`GL-01..GL-26`, `SP-01..SP-30`) with 94.86% union line coverage and 97.37% function coverage across all 35 `src/` contracts. `forge build --sizes` confirms all deployable contracts are strictly within the 24,576-byte EIP-170 runtime limit (`HookRigOrchestrator` at 24,246 bytes; all 12 preset hooks at 24,266–24,268 bytes).

This review was performed by AI agents and automated stateful property fuzzers (Medusa, Echidna, Foundry). Automated analysis and fuzzing can never verify the complete absence of vulnerabilities and no guarantee of security is given. Item statuses reflect an automated re-verification of the code on disk on 2026-10-16 and should be confirmed by a human reviewer. Independent third-party human security audits, bug bounty programs and on-chain monitoring are strongly recommended before mainnet deployment.